DATA 260 – HW 8

7 - Community Sports League Fixtures

SJSU ID : 020053319 | SRI PRAJEET VILLYAL

GitHub repository: https://github.com/villyalsriprajeet-cmd/data260-3319

Tagged commit hash (hw8): 456c2409fe56f6a53c67e1d4e7e36dbc30446860

Hardware: MacBook Pro, Apple M2, 8 GB RAM, macOS

Local model used: qwen2.5:3b through Ollama

Section 0 - Personal Configuration and Domain

Value	Mine
SID4	3319
PORT_BASE	8619
PREFIX	s3319
SEED	3319
VERIFY_SEED	263319
DOMAIN_ID	7 - Community sports league fixtures

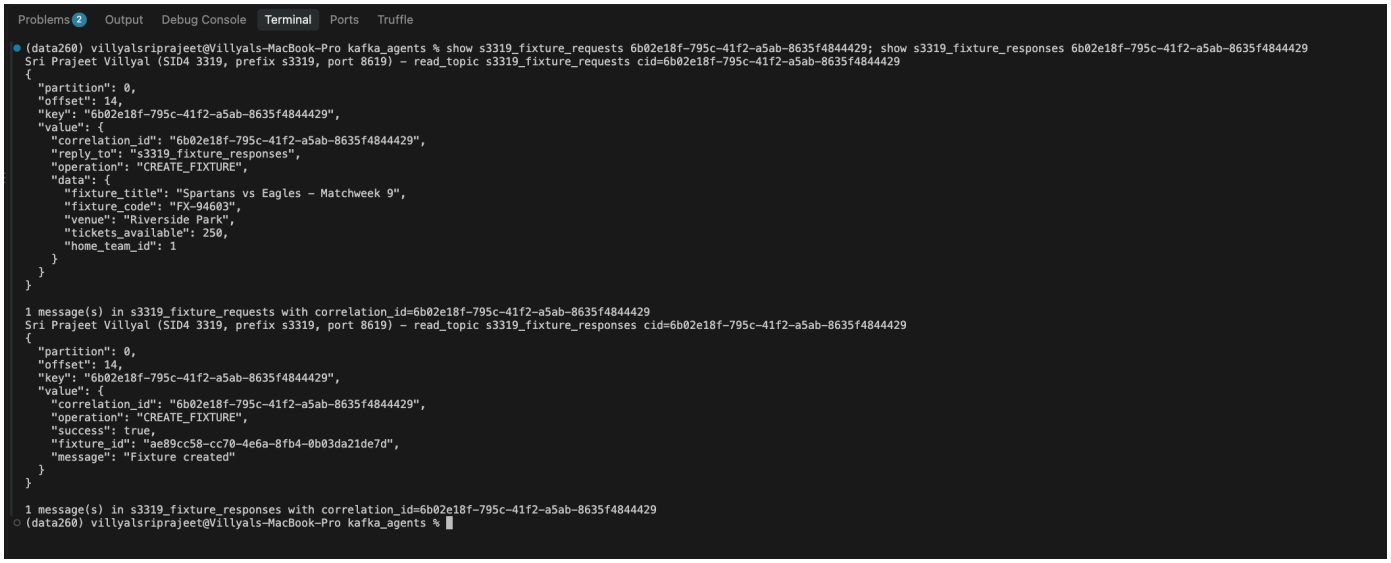
Part 1: Backend Service Request/Response over Kafka

For my domain CREATE_ENTITY is **CREATE_FIXTURE** and GET_ENTITY is **GET_FIXTURE**, with fixture_id as the entity id. FastAPI on port 8619 takes the HTTP call, sends it to Kafka, waits for the reply with the same correlation_id and returns it. The worker is a separate container that consumes requests, validates them, stores the fixture and replies.

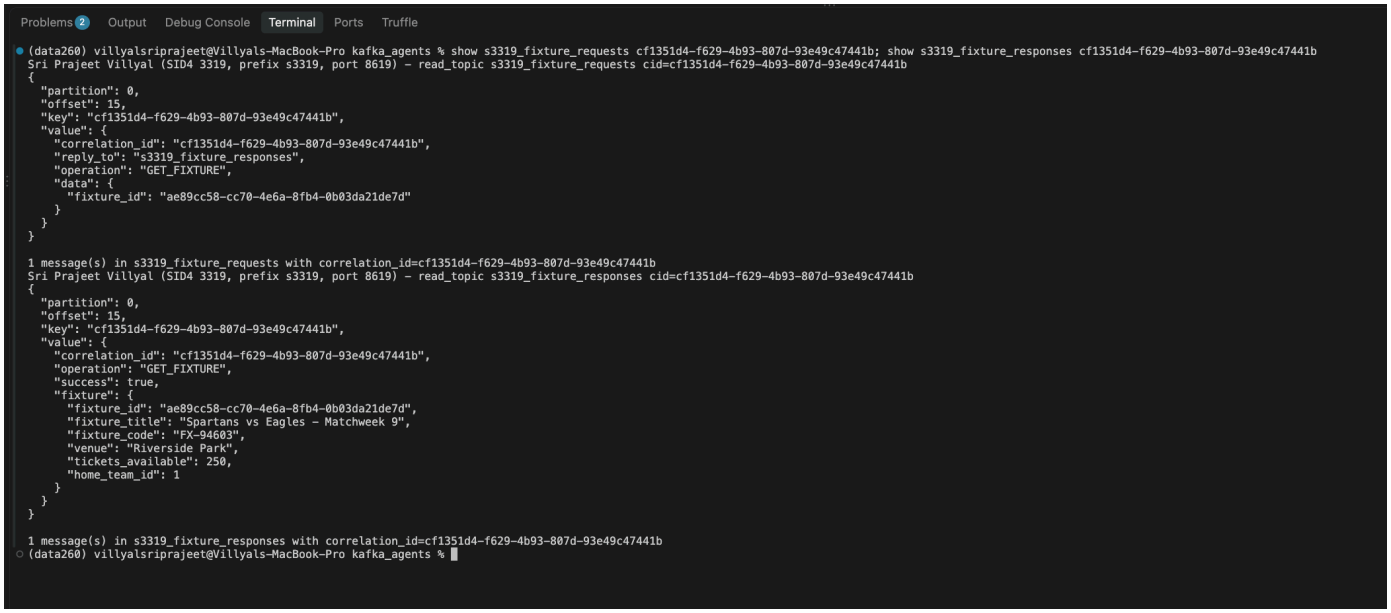
Topics and message schemas

Request topic: **s3319_fixture_requests**. Response topic: **s3319_fixture_responses**. One partition each.

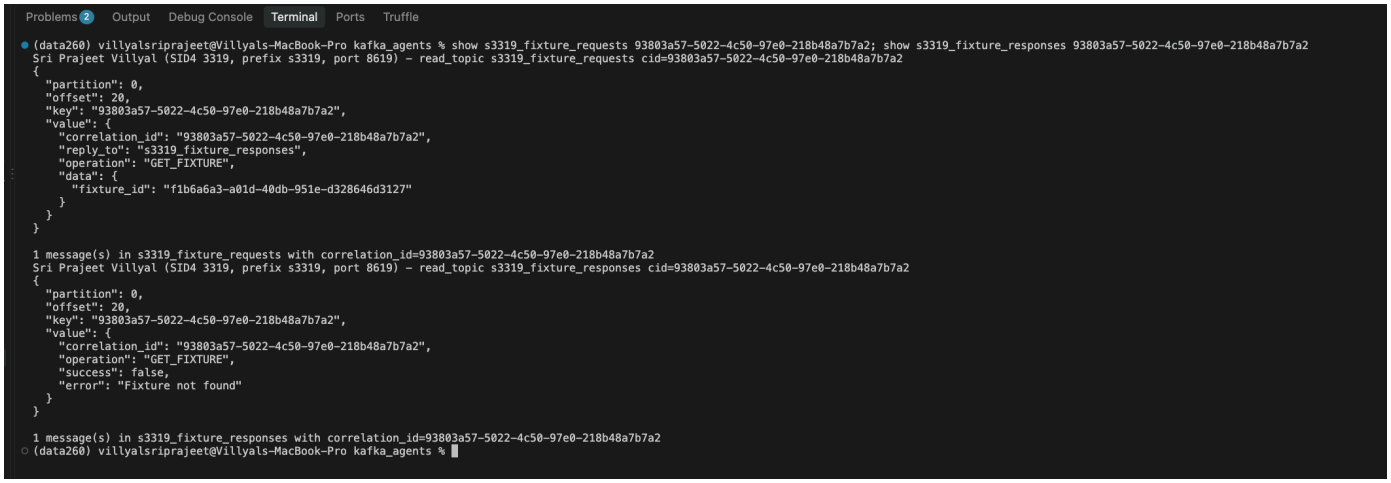
Every request has correlation_id, reply_to, operation and data. Every response has the same correlation_id, the operation, success, and either the result or an error. These are the real messages stored in the two topics during my demo run, printed with read_topic.py.



CREATE_FIXTURE request and response in the two topics.



GET_FIXTURE request and response.



Error response for an unknown fixture_id.

The client makes the correlation_id once. FastAPI and the worker only pass it along, and the worker replies to the topic named in reply_to. The worker commits its offset only after the reply is sent, and a repeated correlation_id gets the stored reply back instead of a second fixture.

Code

code > kafka_service > worker.py > main

```python
def build_response(request):
    cid = request.get("correlation_id") if isinstance(request, dict) else None
    if cid and cid in RESPONSES_BY_CID:
        log("WORKER", "DUPLICATE_SKIPPED", cid, note="same correlation_id seen before, resending stored reply")
        return {**RESPONSES_BY_CID[cid], "duplicate": True}
    response = process_request(request, FIXTURES)
    if cid:
        RESPONSES_BY_CID[cid] = response
    return response

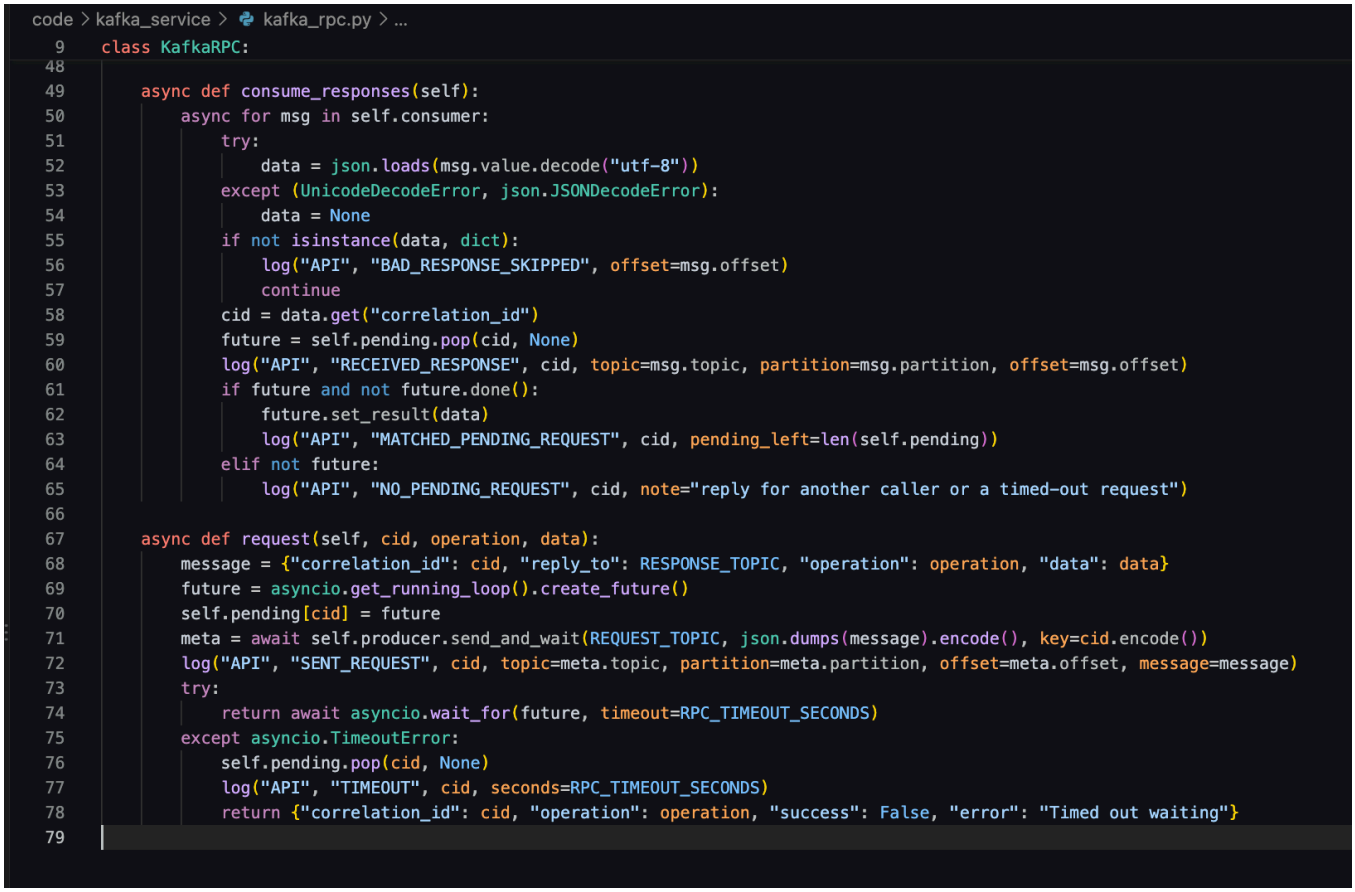

async def handle(msg, producer, consumer):
    request = decode(msg.value)
    cid = request.get("correlation_id") if isinstance(request, dict) else None
    log("WORKER", "RECEIVED", cid, topic=msg.topic, partition=msg.partition, offset=msg.offset, message=request)
    response = build_response(request)
    log("WORKER", "PROCESSED", cid, operation=response.get("operation"), success=response["success"])
    target = reply_topic(request)
    meta = await producer.send_and_wait(target, json.dumps(response).encode(), key=(cid or "").encode() or None)
    log(
        "WORKER", "SENT_RESPONSE", cid, topic=meta.topic, partition=meta.partition, offset=meta.offset, message=response
    )
    await consumer.commit({TopicPartition(msg.topic, msg.partition): msg.offset + 1})
    log("WORKER", "OFFSET_COMMITTED", cid, partition=msg.partition, next_offset=msg.offset + 1)

```

worker.py: process the request, reply, then commit.



fixture_store.py: validation, CREATE_FIXTURE and GET_FIXTURE.



kafka_rpc.py: FastAPI waits for the reply with the same correlation_id.



```
code > kafka_service > api.py > ...
14  class FixtureRequest(BaseModel):
15      correlation_id: str | None = Field(default=None, max_length=100)  # sent by the client
16      operation: str = Field(min_length=1)
17      data: dict = Field(default_factory=dict)
18
19
20  @asynccontextmanager
21  async def lifespan(app):
22      banner("Part 1 FastAPI service")
23      await RPC.start()
24      log("API", "SERVER_STARTED", url=f"http://127.0.0.1:{PORT_BASE}")
25      print(f"FastAPI server started at http://127.0.0.1:{PORT_BASE}", flush=True)
26      yield
27      await RPC.stop()
28
29
30  APP = FastAPI(title=f"{PREFIX} Fixture service over Kafka - HW8", lifespan=lifespan)
31
32
33  @APP.get("/")
34  async def home():
35      return {
36          "service": "fixture request/response over Kafka",
37          "student": STUDENT_NAME,
38          "sid4": SID4,
39          "request_topic": REQUEST_TOPIC,
40          "response_topic": RESPONSE_TOPIC,
41      }
42
43
44  @APP.post("/fixtures/kafka")
45  async def fixture_operation(body: FixtureRequest):
46      cid = body.correlation_id
47      if not cid:
48          cid = str(uuid.uuid4())  # only if the client did not send one
49          log("API", "CORRELATION_ID_CREATED_BY_API", cid)
50      log("API", "HTTP_REQUEST_RECEIVED", cid, operation=body.operation, data=body.data)
51      response = await RPC.request(cid, body.operation, body.data)
52      log("API", "RETURNING_HTTP_RESPONSE", cid, success=response.get("success"), response=response)
53      status = 504 if response.get("error") == "Timed out waiting" else 200
54      return JSONResponse(status_code=status, content=response)
55
```

api.py: FastAPI on port 8619.

```
code > homework_demo.py > show_sequence
58  def call(operation, data, since):
59      cid = str(uuid.uuid4())  # made once by the client
60      body = {"correlation_id": cid, "operation": operation, "data": data}
61      sent = client_line("SENDING_HTTP_REQUEST", cid, body)
62      print(sent)
63      response = requests.post(API_URL, json=body, timeout=30).json()
64      got = client_line("RECEIVED_RESPONSE", cid, response)
65      print(got)
66      time.sleep(0.5)  # give docker time to write the logs
67      lines = container_lines(since, cid)
68      cut = next((i + 1 for i, ln in enumerate(lines) if "RETURNING_HTTP_RESPONSE" in ln), len(lines))
69      sequence = [sent] + lines[:cut] + [got] + lines[cut:]  # client gets the reply right after the API returns it
70      return cid, response, sequence
71
```

homework_demo.py: one correlation_id per request.

```python
code > homework_demo.py > show_sequence
def main():
    banner("Part 1 homework_demo.py")
    since = (datetime.now(timezone.utc) - timedelta(seconds=5)).strftime("%Y-%m-%dT%H:%M:%SZ")
    print(requests.get(API_URL.rsplit("/fixtures", 1)[0] + "/", timeout=10).json())
    fixture = new_fixture()
    print("\nSteps 1-6: CREATE_FIXTURE")
    create = call("CREATE_FIXTURE", fixture, since)
    show_sequence("CREATE_FIXTURE", create[2])
    fixture_id = create[1].get("fixture_id")  # id made by the worker
    print(f"\nStep 7: generated fixture_id = {fixture_id}")
    print("\nSteps 8-9: GET_FIXTURE")
    get = call("GET_FIXTURE", {"fixture_id": fixture_id}, since)
    show_sequence("GET_FIXTURE", get[2])
    passed = step_ten(fixture, create, get)
    errors = error_cases(since, fixture)
    result = {
        "create": {"correlation_id": create[0], "request_data": fixture, "response": create[1]},
        "get": {"correlation_id": get[0], "response": get[1]},
        "error_cases": errors,
        "round_trip_passed": passed,
    }
    save(result, create[2] + [""] + get[2])
    ok = sum(e["passed"] for e in errors)
    print(f"\nRESULT: CREATE -> GET round trip {'PASSED' if passed else 'FAILED'}; {ok}/{len(errors)} clean errors")
    banner("end of homework_demo.py")


if __name__ == "__main__":
    main()
```

homework_demo.py: CREATE, GET, compare, error cases.

Docker containers, worker and API

```
Problems  Output  Debug Console  Terminal  Ports  Truffle

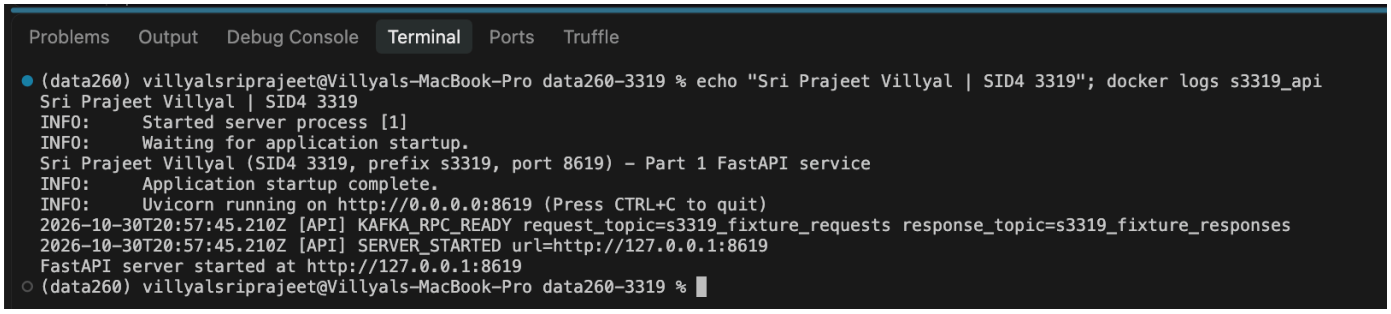
(data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % echo "Sri Prajeet Villyal | SID4 3319"; docker ps
Sri Prajeet Villyal | SID4 3319
CONTAINER ID   IMAGE                COMMAND                  CREATED          STATUS                    PORTS                    NAMES
37ef0053b2b1   data260-3319-api     "uvicorn api:APP --h…"   6 minutes ago    Up 6 minutes              127.0.0.1:8619->8619/tcp   s3319_api
27b706526892   data260-3319-worker  "python worker.py"       6 minutes ago    Up 6 minutes              8619/tcp                 s3319_worker
e382637e91ef   apache/kafka:3.9.2   "/__cacert_entrypoin…"   13 minutes ago   Up 13 minutes (healthy)   0.0.0.0:9092->9092/tcp   s3319_kafka
(data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 %
```

Kafka, worker and API containers running.

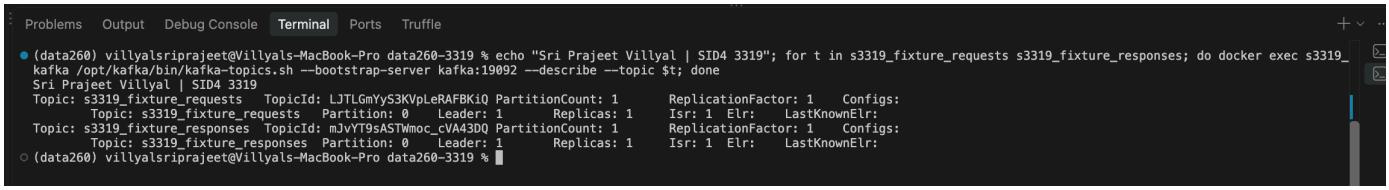
```
Problems  Output  Debug Console  Terminal  Ports  Truffle

(data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % echo "Sri Prajeet Villyal | SID4 3319"; docker logs s3319_worker
Sri Prajeet Villyal | SID4 3319
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) - Part 1 worker (consumer)
Consumer is ready and listening...
2026-10-30T20:57:47.761Z [WORKER] LISTENING topic=s3319_fixture_requests group=s3319_fixture_worker_group bootstrap=kafka:19092
(data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 %
```

Worker log: "Consumer is ready and listening."



API log: "FastAPI server started at http://127.0.0.1:8619"

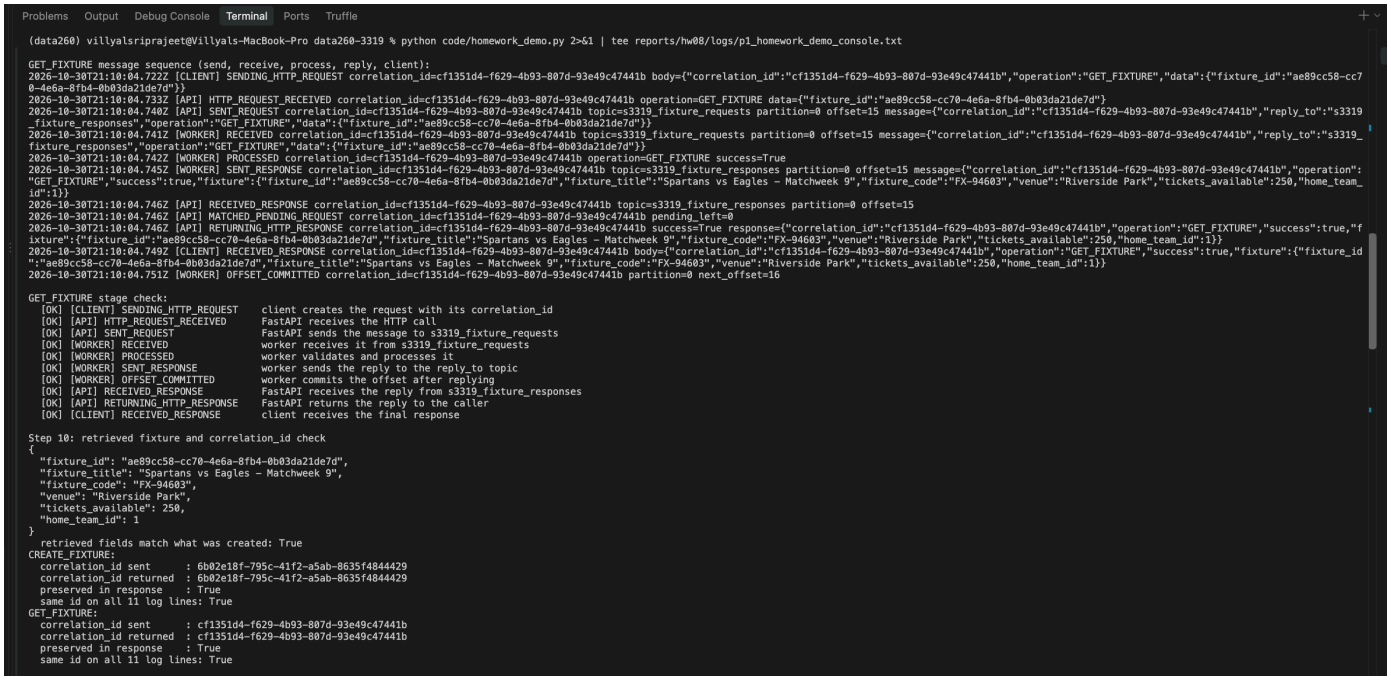


The two Part 1 topics.

Message flow and full CREATE then GET sequence



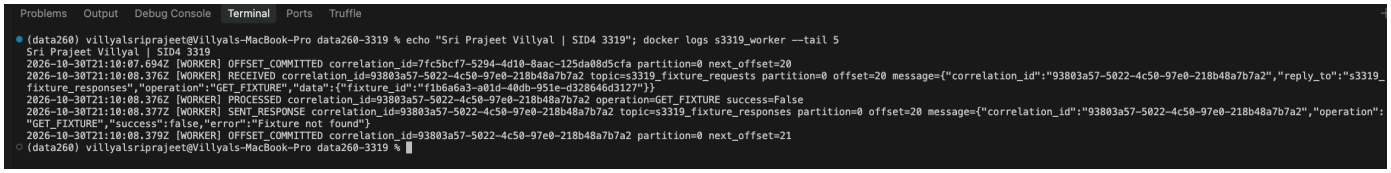
CREATE_FIXTURE: all 10 stages in order.



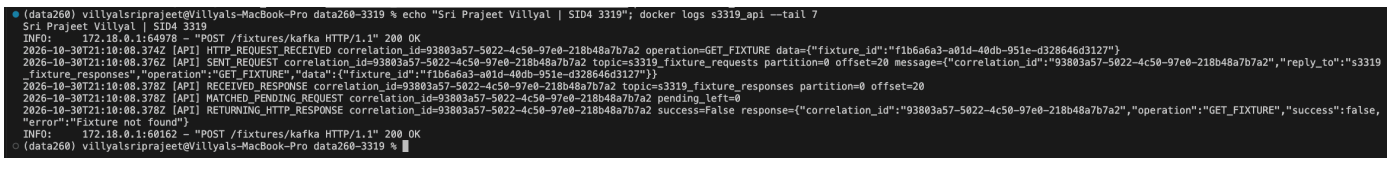
GET_FIXTURE: same fixture back, correlation_id kept.



Error cases: 5/5 clean errors, round trip passed.



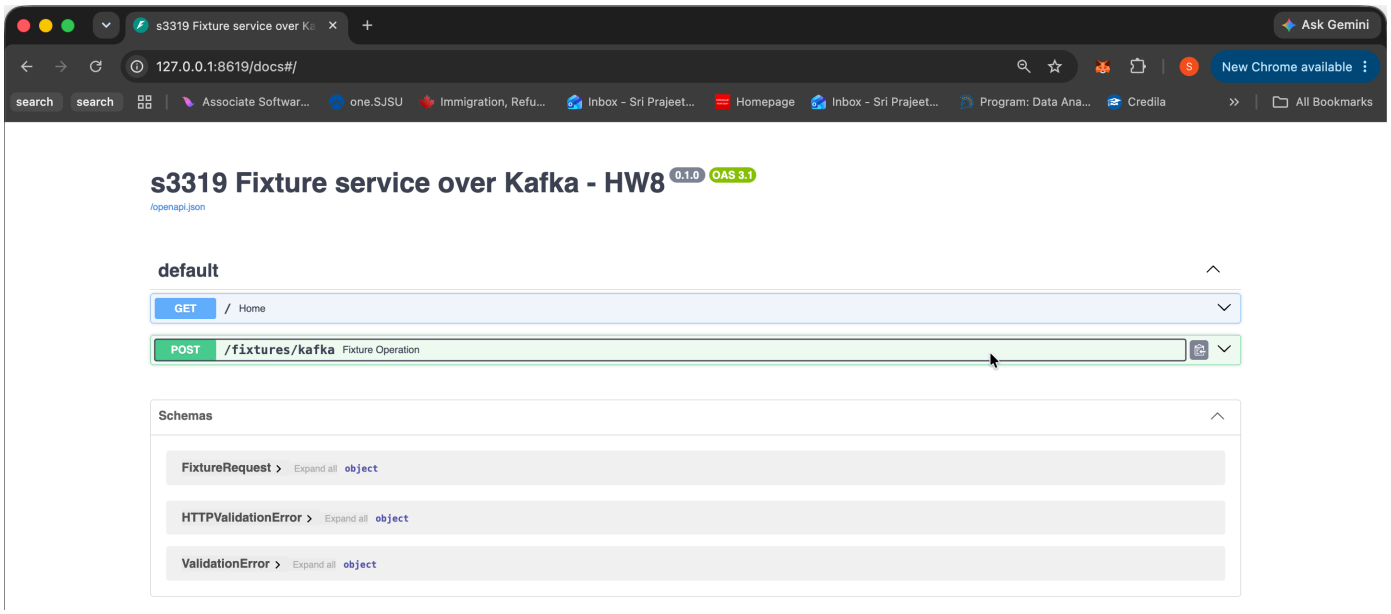
Worker log for one request.



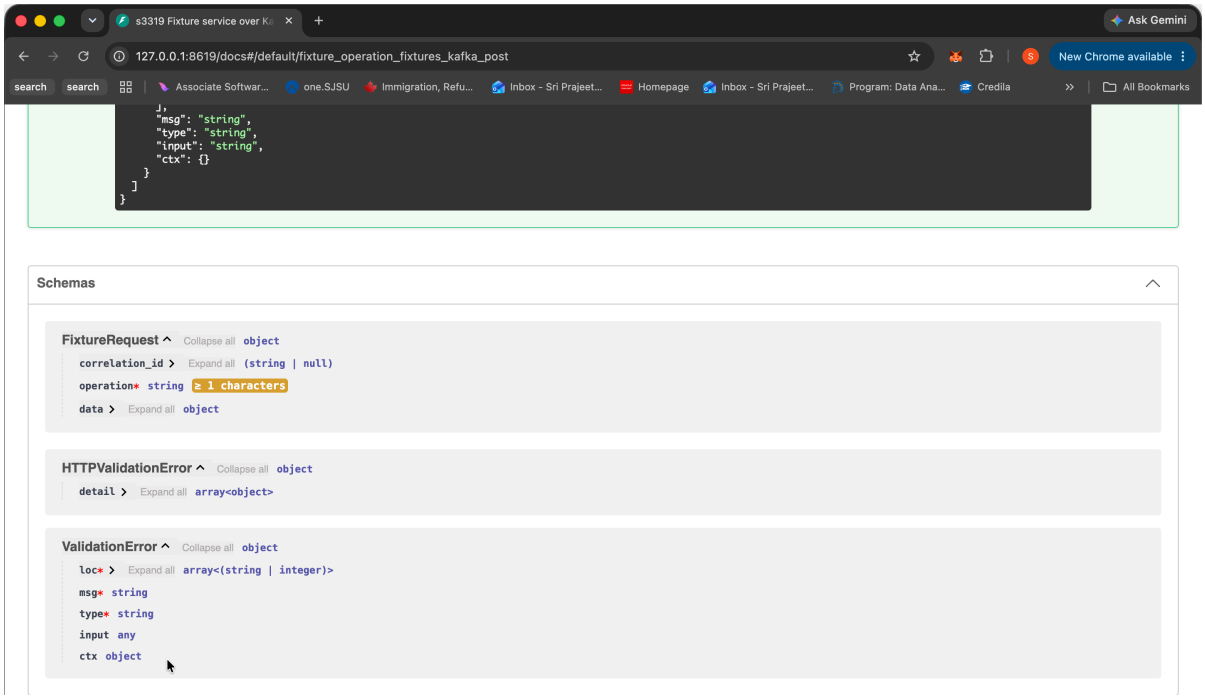
API log for the same request.

API calls through FastAPI (Postman and Swagger)

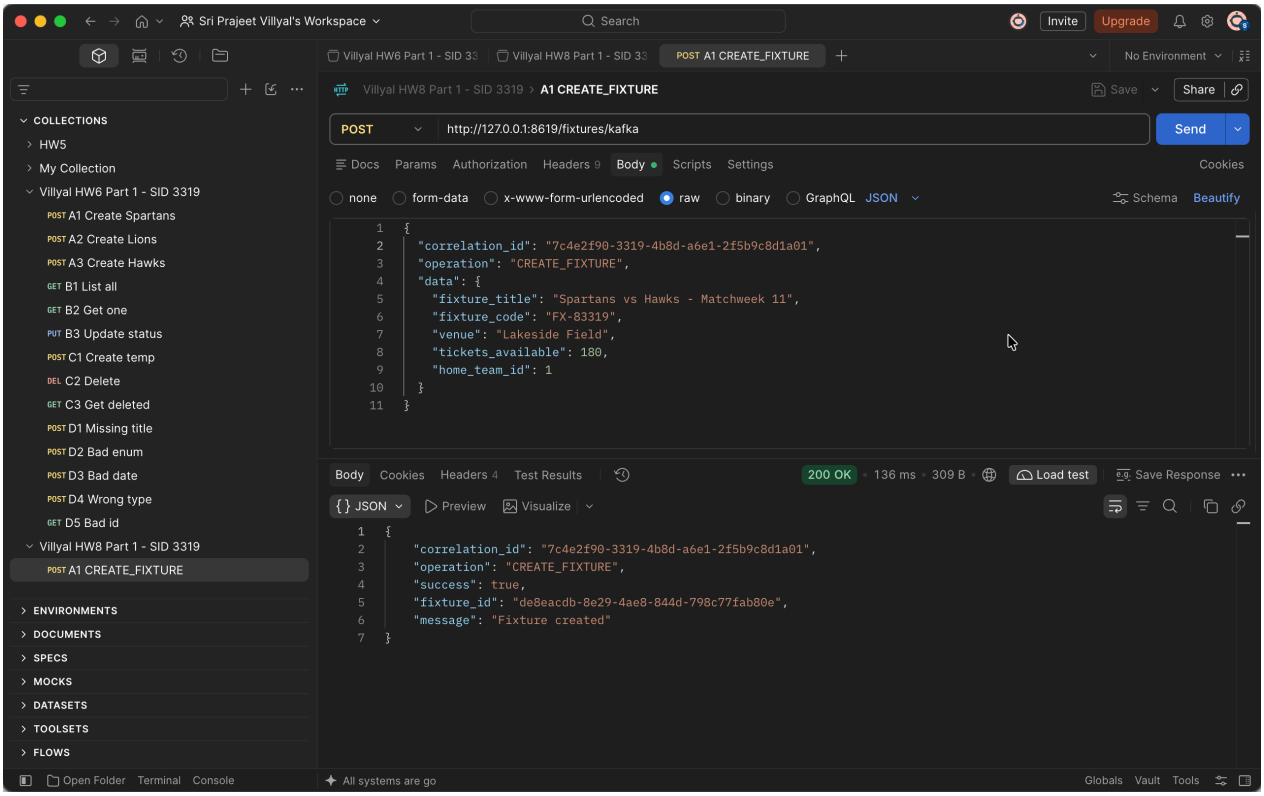
The same service can also be called directly over HTTP. FastAPI publishes its OpenAPI page at /docs, and I sent CREATE, GET and an error case from Postman to POST http://127.0.0.1:8619/fixtures/kafka. FastAPI adds reply_to and forwards each call through Kafka to the worker, so every reply here came back over s3319_fixture_responses.



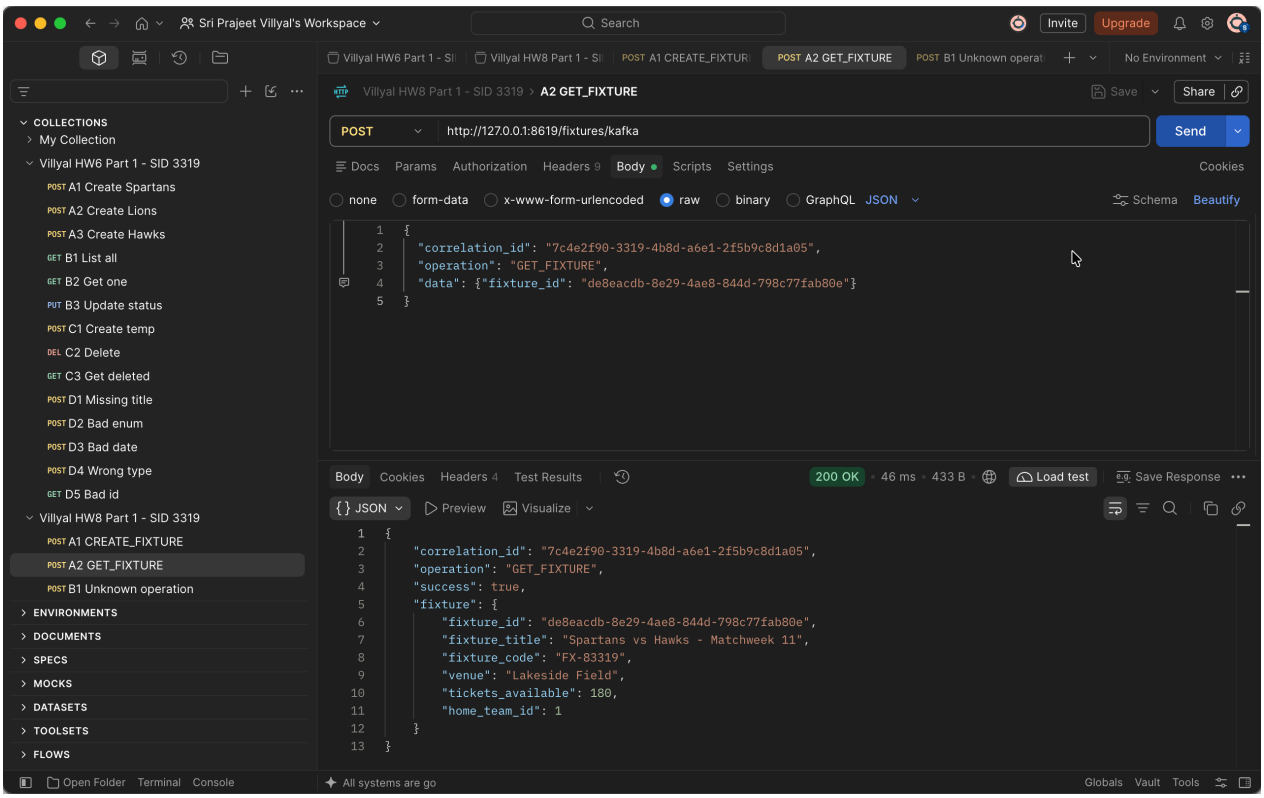
Swagger page for the API.



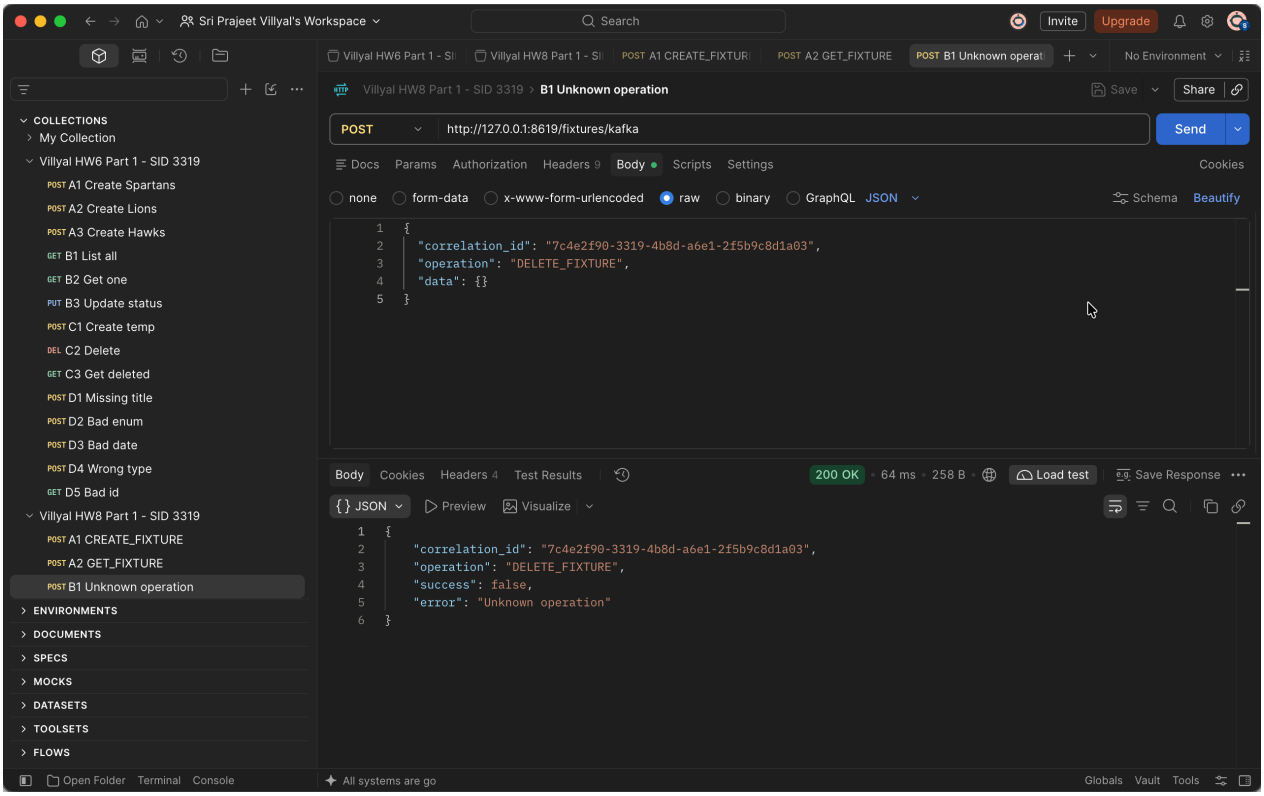
FixtureRequest schema.



Postman CREATE_FIXTURE: 200 OK, fixture created.



Postman GET_FIXTURE: 200 OK, full fixture.



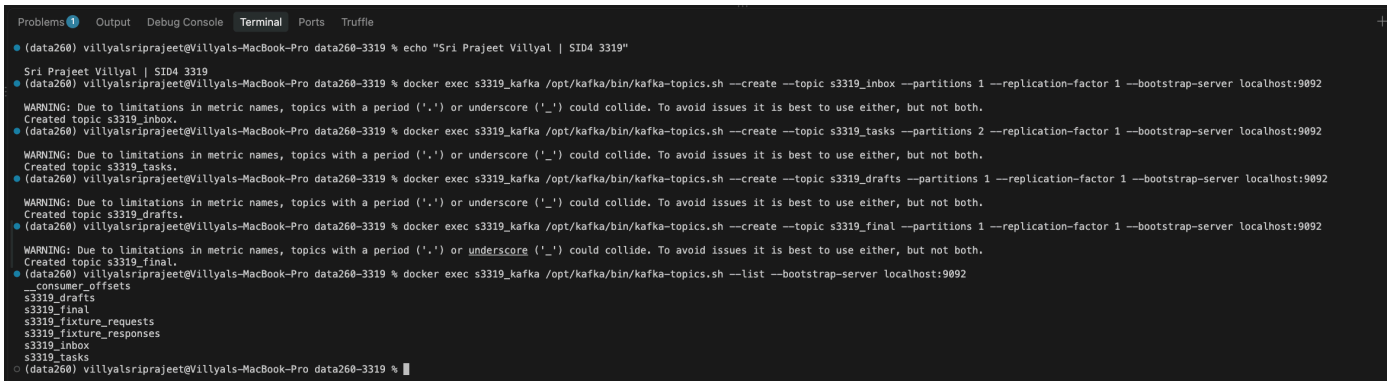
Postman unknown operation: clean error.

Note: the API and worker run in Docker and the demo client runs on the Mac, so the timestamps come from two clocks and can be a few ms apart. That is why the sequence in the demo is ordered by stage when two lines have the same time.

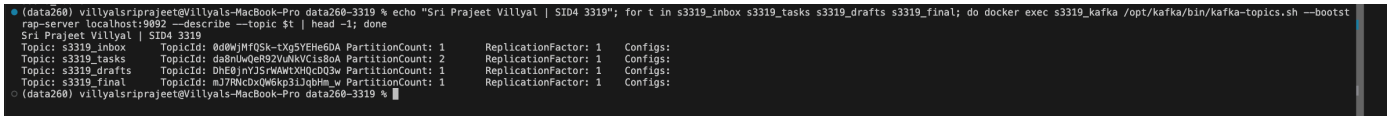
Part 2: Three Agent Kafka Pipeline

Planner, Writer and Reviewer run as three separate Python processes. They never call each other. Each one reads a topic, calls qwen2.5:3b and writes to the next topic, keyed by the correlation_id.

Sender	Reads topic	Sends to
Me (send_question.py)	-	s3319_inbox (a question)
Planner	s3319_inbox	s3319_tasks (a 3 step plan)
Writer	s3319_tasks (2 partitions)	s3319_drafts (a draft answer)
Reviewer	s3319_drafts	s3319_final (status and answer)

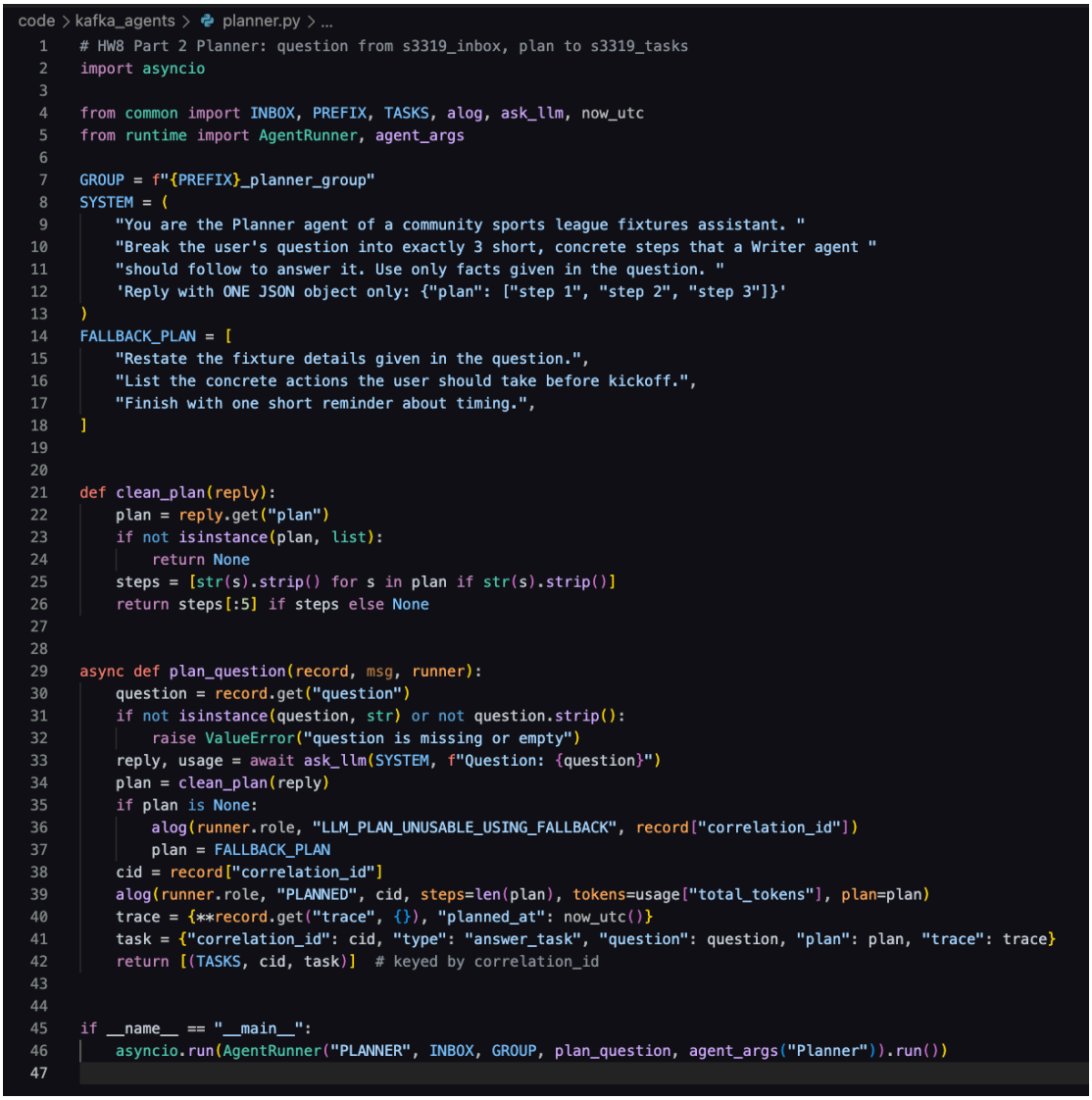


Creating the four topics.

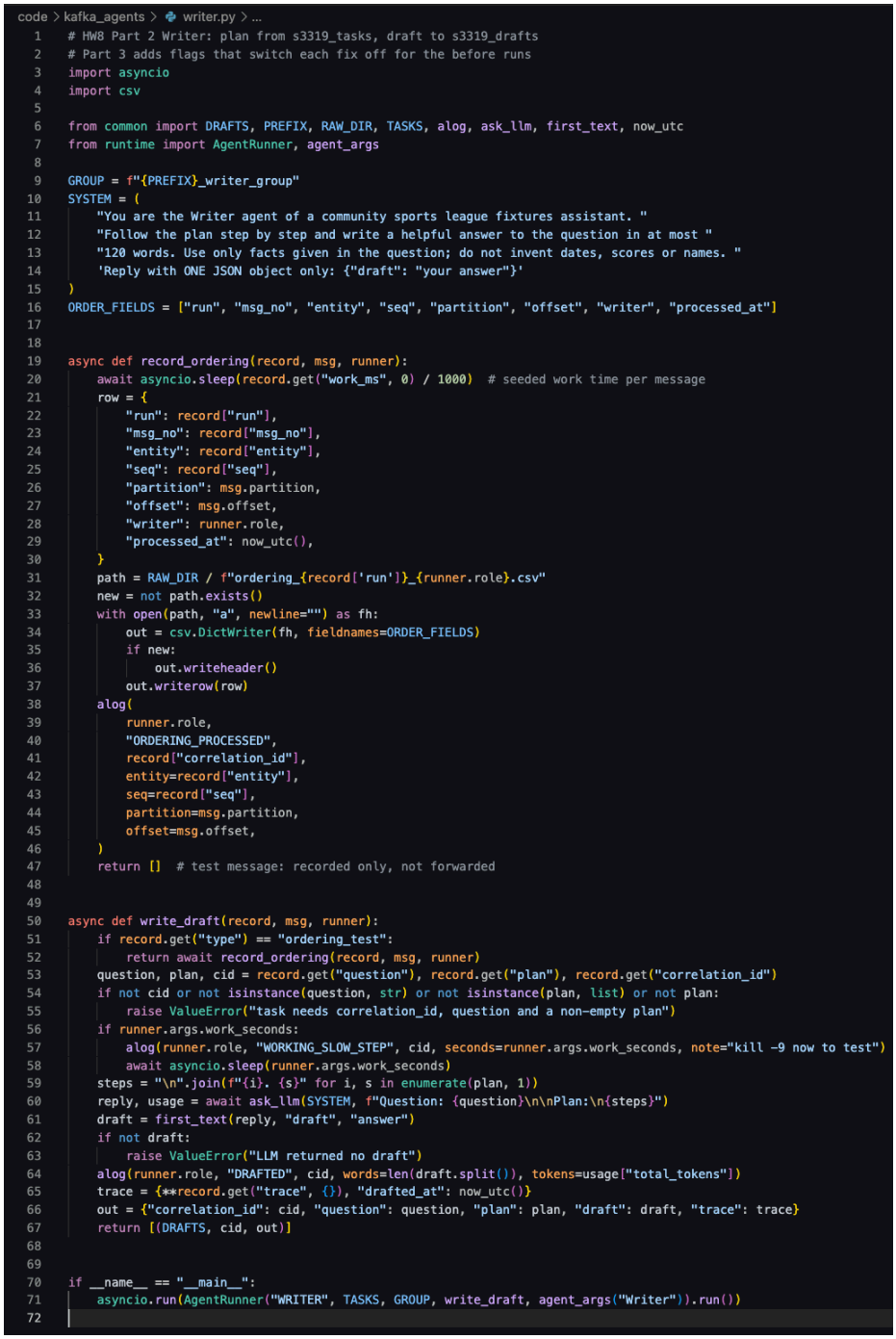


Partition counts (tasks has 2).

Agent code



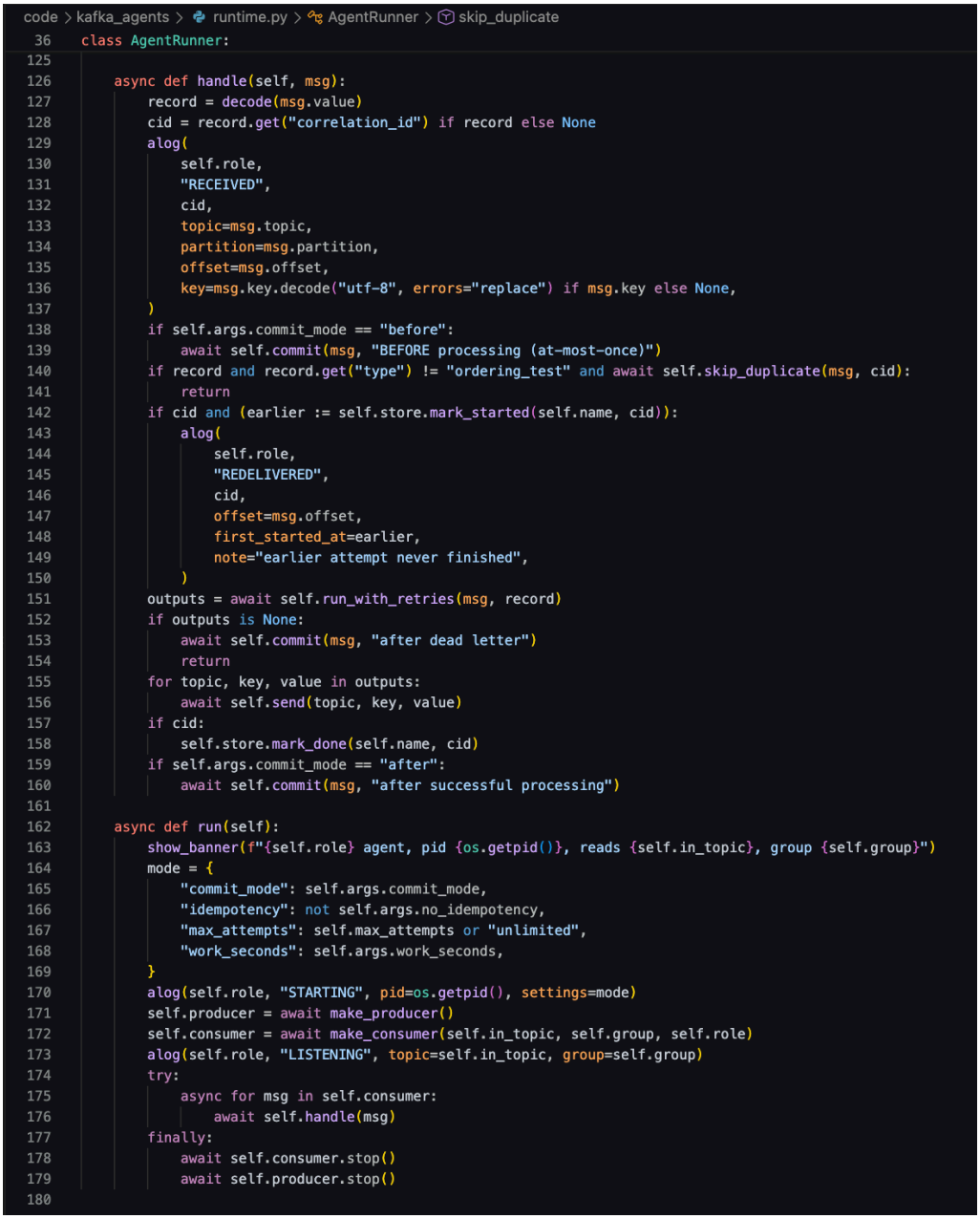
planner.py


code > kafka_agents > writer.py > ...
HW8 Part 2 Writer: plan from s3319_tasks, draft to s3319_drafts
Part 3 adds flags that switch each fix off for the before runs
import asyncio
import csv
from common import DRAFTS, PREFIX, RAW_DIR, TASKS, alog, ask_llm, first_text, now_utc
from runtime import AgentRunner, agent_args
GROUP = f"{PREFIX}_writer_group"
SYSTEM = (
"You are the Writer agent of a community sports league fixtures assistant. "
"Follow the plan step by step and write a helpful answer to the question in at most "
"120 words. Use only facts given in the question; do not invent dates, scores or names. "
'Reply with ONE JSON object only: {"draft": "your answer"}'
)
ORDER_FIELDS = ["run", "msg_no", "entity", "seq", "partition", "offset", "writer", "processed_at"]
async def record_ordering(record, msg, runner):
await asyncio.sleep(record.get("work_ms", 0) / 1000) # seeded work time per message
row = {
"run": record["run"],
"msg_no": record["msg_no"],
"entity": record["entity"],
"seq": record["seq"],
"partition": msg.partition,
"offset": msg.offset,
"writer": runner.role,
"processed_at": now_utc(),
}
path = RAW_DIR / f"ordering_{record['run']}_{runner.role}.csv"
new = not path.exists()
with open(path, "a", newline="") as fh:
out = csv.DictWriter(fh, fieldnames=ORDER_FIELDS)
if new:
out.writeheader()
out.writerow(row)
alog(
runner.role,
"ORDERING_PROCESSED",
record["correlation_id"],
entity=record["entity"],
seq=record["seq"],
partition=msg.partition,
offset=msg.offset,
)
return [] # test message: recorded only, not forwarded
async def write_draft(record, msg, runner):
if record.get("type") == "ordering_test":
return await record_ordering(record, msg, runner)
question, plan, cid = record.get("question"), record.get("plan"), record.get("correlation_id")
if not cid or not isinstance(question, str) or not isinstance(plan, list) or not plan:
raise ValueError("task needs correlation_id, question and a non-empty plan")
if runner.args.work_seconds:
alog(runner.role, "WORKING_SLOW_STEP", cid, seconds=runner.args.work_seconds, note="kill -9 now to test")
await asyncio.sleep(runner.args.work_seconds)
steps = "\n".join(f"{i}. {s}" for i, s in enumerate(plan, 1))
reply, usage = await ask_llm(SYSTEM, f"Question: {question}\n\nPlan:\n{steps}")
draft = first_text(reply, "draft", "answer")
if not draft:
raise ValueError("LLM returned no draft")
alog(runner.role, "DRAFTED", cid, words=len(draft.split()), tokens=usage["total_tokens"])
trace = {**record.get("trace", {}), "drafted_at": now_utc()}
out = {"correlation_id": cid, "question": question, "plan": plan, "draft": draft, "trace": trace}
return [(DRAFTS, cid, out)]
if __name__ == "__main__":
asyncio.run(AgentRunner("WRITER", TASKS, GROUP, write_draft, agent_args("Writer")).run())

writer.py


code > kafka_agents > reviewer.py > ...
HW8 Part 2 Reviewer: draft from s3319_drafts, answer to s3319_final
import asyncio
from common import DRAFTS, FINAL, PREFIX, alog, ask_llm, now_utc
from runtime import AgentRunner, agent_args
GROUP = f"{PREFIX}_reviewer_group"
MAX_WORDS = 150
SYSTEM = (
"You are the Reviewer agent of a community sports league fixtures assistant. "
"Check that the draft answers the question, follows the plan, is at most 150 words and "
"invents no facts that are not in the question. Fix small problems yourself. "
'Reply with ONE JSON object only: {"status": "approved" or "needs_revision", '
'"answer": "the final answer text", "notes": "one sentence"}'
)
def decide(reply, draft):
given = reply.get("answer")
answer = given.strip() if isinstance(given, str) and given.strip() else draft.strip() # fall back to draft
status = "approved" if str(reply.get("status", "")).lower() == "approved" else "needs_revision"
checks = {"answer_not_empty": bool(answer), "within_word_limit": len(answer.split()) <= MAX_WORDS}
if not all(checks.values()):
status = "needs_revision" # code checks can veto the model's approval
return status, answer, checks
async def review_draft(record, msg, runner):
cid, draft, question = record.get("correlation_id"), record.get("draft"), record.get("question")
if not cid or not isinstance(draft, str) or not draft.strip():
raise ValueError("draft message needs correlation_id and a draft")
user = f"Question: {question}\n\nPlan: {record.get('plan')}\n\nDraft: {draft}"
reply, usage = await ask_llm(SYSTEM, user)
status, answer, checks = decide(reply, draft)
alog(runner.role, "REVIEWED", cid, status=status, tokens=usage["total_tokens"], checks=checks)
trace = {**record.get("trace", {}), "reviewed_at": now_utc()}
final = {
"correlation_id": cid,
"status": status,
"answer": answer,
"question": question,
"review_notes": str(reply.get("notes", "")).strip(),
"checks": checks,
"trace": trace,
}
return [(FINAL, cid, final)]
if __name__ == "__main__":
asyncio.run(AgentRunner("REVIEWER", DRAFTS, GROUP, review_draft, agent_args("Reviewer")).run())

reviewer.py



runtime.py: shared consume loop, commit after output.

code > kafka_agents > send_question.py > ...

```python

async def wait_for_final(consumer, cid, timeout):
    async def find():
        async for msg in consumer:
            record = decode(msg.value)
            if record and record.get("correlation_id") == cid:
                alog("CLIENT", "FINAL_RECEIVED", cid, topic=msg.topic, partition=msg.partition, offset=msg.offset)
                return record

    return await asyncio.wait_for(find(), timeout=timeout)

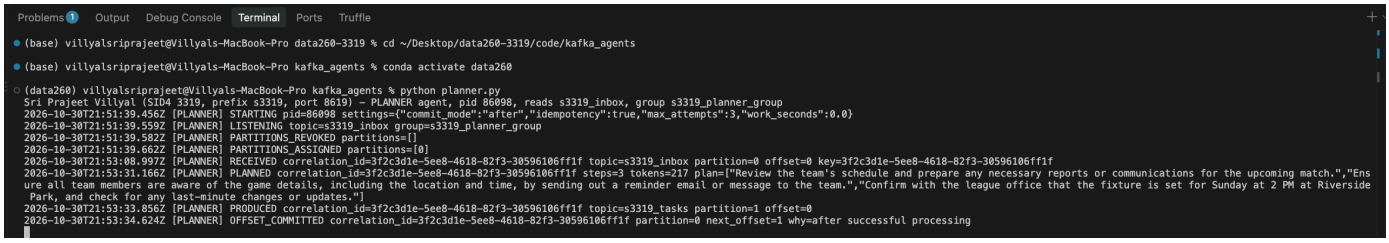

async def main(args):
    show_banner("Part 2 send_question.py")
    cid = str(uuid.uuid4())
    message = {"correlation_id": cid, "question": args.question, "trace": {"asked_at": now_utc()}}
    consumer = None if args.no_wait else await tail_consumer(FINAL)  # listen before sending
    producer = await make_producer()
    meta = await producer.send_and_wait(INBOX, encode(message), key=cid.encode())
    await producer.stop()
    alog(
        "CLIENT", "QUESTION_SENT", cid, topic=meta.topic, partition=meta.partition, offset=meta.offset, message=message
    )
    if consumer is None:
        return
    try:
        final = await wait_for_final(consumer, cid, args.timeout)
    finally:
        await consumer.stop()
    print(f"\nFinal JSON from {FINAL}:\n{json.dumps(final, indent=2)}")
    print(f"\ncorrelation_id preserved end to end: {final.get('correlation_id') == cid}")
    print("\nMessage journey (all agents, same correlation_id):")
    print("\n".join(journey_lines(cid)))
    RAW_DIR.mkdir(parents=True, exist_ok=True)
    (RAW_DIR / "p2_final_message.json").write_text(json.dumps(final, indent=2) + "\n")
    show_banner("end of send_question.py")


if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--question", default=QUESTION)
    parser.add_argument("--no-wait", action="store_true")
    parser.add_argument("--timeout", type=float, default=600)
    asyncio.run(main(parser.parse_args()))

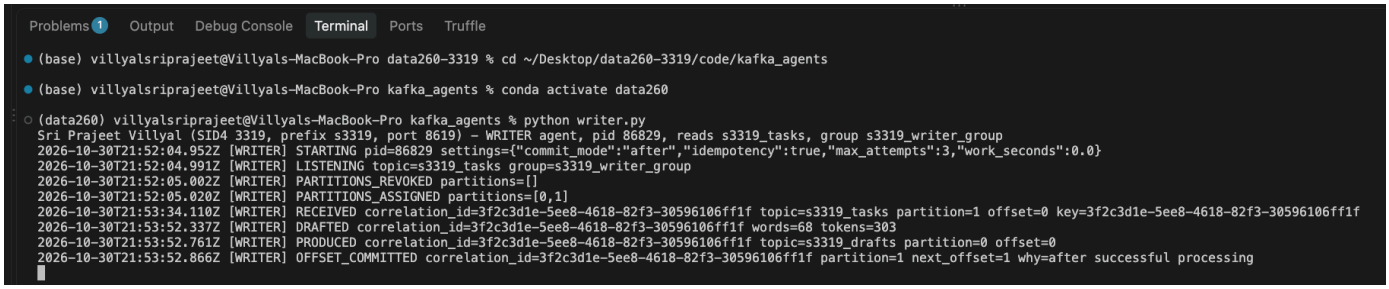
```

send_question.py

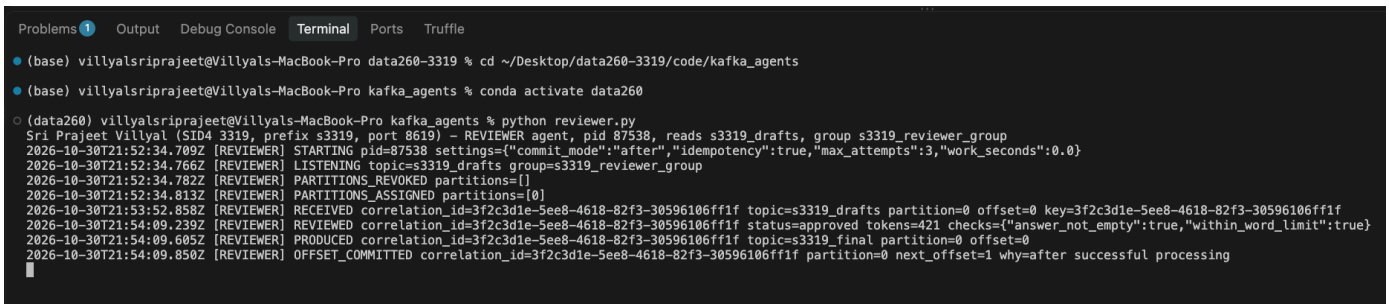
Agents running



Planner run.

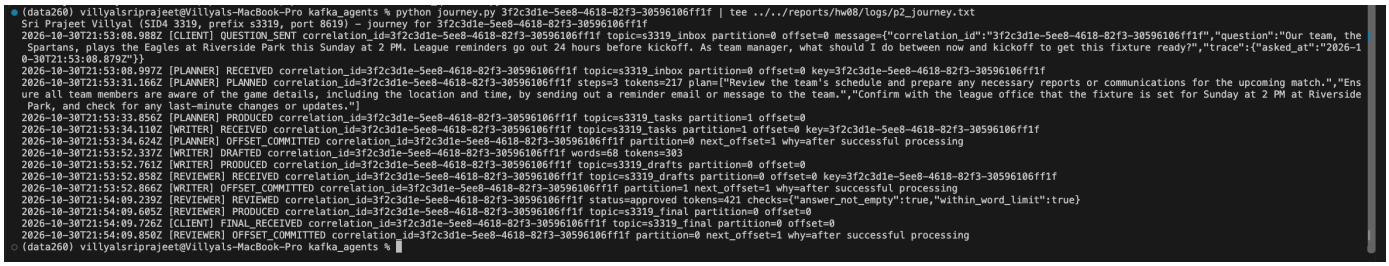


Writer run.

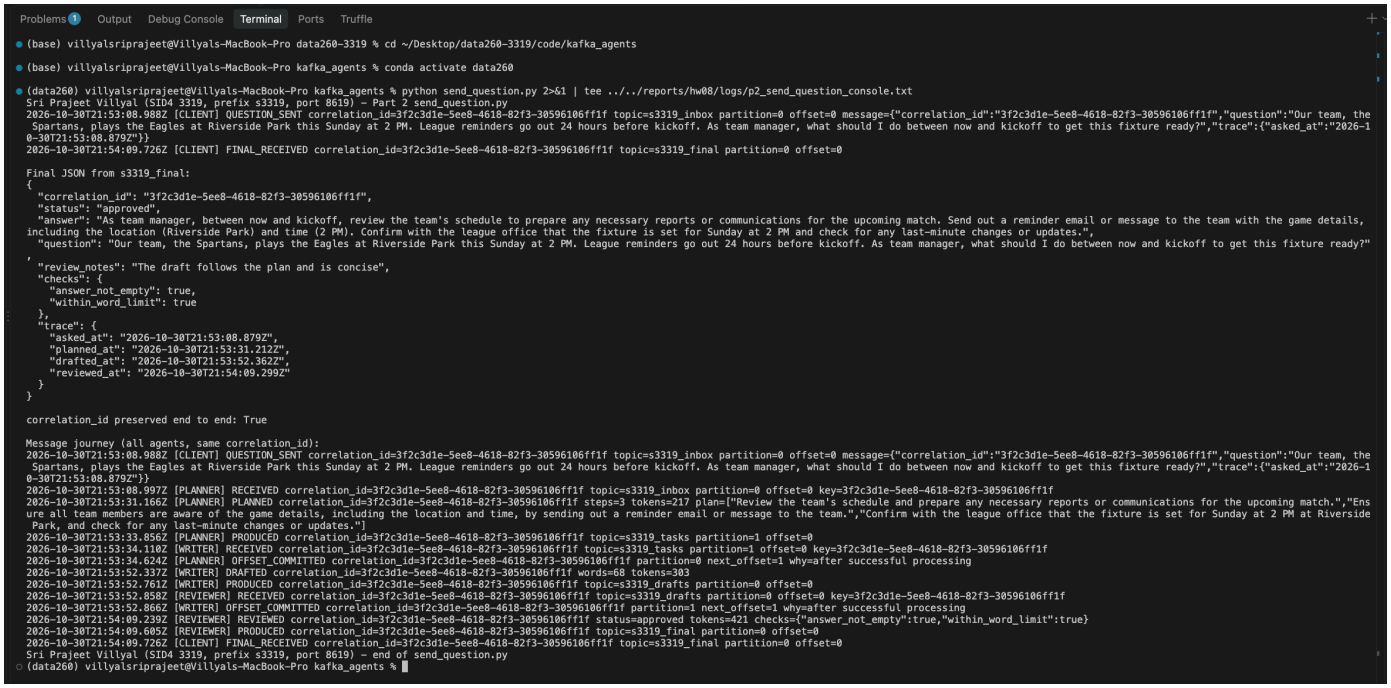


Reviewer run.

One full message journey and the final JSON



Full journey from inbox to final, one correlation_id.



Final JSON with status and answer.

How Kafka helped the agents coordinate

The agents only know topic names, not each other. The Planner does not wait for the Writer, it just writes a task and moves on. Each message stays in its topic with an offset, so if an agent is slow or stopped the work waits there instead of being lost. The correlation_id on every message and log line let me follow one question across three processes, which is how I built the journey above. The whole run took about 61 seconds: Planner 22.2 s, Writer 18.2 s and Reviewer 16.4 s.

Part 3: Failure Behavior

The Writer has flags that switch each fix off, so the same code can show the bug (before) and the fix (after). All the before runs use these flags, and all the after runs use the default Writer.

```python
def agent_args(name):
    parser = argparse.ArgumentParser(description=f"{name} agent (HW8)")
    parser.add_argument("--instance", default="", help="label when running two copies, e.g. A or B")
    parser.add_argument(
        "--commit-mode",
        choices=["after", "before"],
        default="after",
        help="after = commit only after success (fix); before = commit on receipt (bug)",
    )
    parser.add_argument("--no-idempotency", action="store_true", help="process replays again (bug)")
    parser.add_argument("--no-dlq", action="store_true", help="retry a failing message forever (bug)")
    parser.add_argument("--work-seconds", type=float, default=0.0, help="extra slow step, for the kill test")
    return parser.parse_args()


```

Flags used for the before runs.

I. Duplicate delivery

I replayed the task for 3f2c3d1e onto s3319_tasks with the same key, value and correlation_id.

```python
async def main(args):
    show_banner(f"Part 3 duplicate replay of {args.cid} on {args.topic}")
    msgs = [m for m in await read_all(args.topic) if (decode(m.value) or {}).get("correlation_id") == args.cid]
    if not msgs:
        print(f"no message with correlation_id={args.cid} in {args.topic}")
        return
    original = msgs[0]
    alog("REPLAY", "ORIGINAL_FOUND", args.cid, topic=args.topic, partition=original.partition, offset=original.offset)
    producer = await make_producer()
    meta = await producer.send_and_wait(args.topic, original.value, key=original.key)  # byte-for-byte copy
    await producer.stop()
    alog(
        "REPLAY",
        "DUPLICATE_SENT",
        args.cid,
        topic=meta.topic,
        partition=meta.partition,
        offset=meta.offset,
        note="same key, same value, same correlation_id",
    )
```

replay_message.py: re-sends the same message.

```python
class ProcessedStore:
    def __init__(self, path=STATE_DB):
        path.parent.mkdir(parents=True, exist_ok=True)
        self.db = sqlite3.connect(path, timeout=10)
        self.db.execute("CREATE TABLE IF NOT EXISTS started (agent TEXT, cid TEXT, at TEXT, PRIMARY KEY (agent, cid))")
        self.db.execute("CREATE TABLE IF NOT EXISTS done (agent TEXT, cid TEXT, at TEXT, PRIMARY KEY (agent, cid))")
        self.db.commit()

    def done_at(self, agent, cid):
        row = self.db.execute("SELECT at FROM done WHERE agent=? AND cid=?", (agent, cid)).fetchone()
        return row[0] if row else None

    def mark_started(self, agent, cid):
        row = self.db.execute("SELECT at FROM started WHERE agent=? AND cid=?", (agent, cid)).fetchone()
        self.db.execute("INSERT OR IGNORE INTO started VALUES (?, ?, ?)", (agent, cid, now_utc()))
        self.db.commit()
        return row[0] if row and not self.done_at(agent, cid) else None  # started before, never finished

    def mark_done(self, agent, cid):
        self.db.execute("INSERT OR IGNORE INTO done VALUES (?, ?, ?)", (agent, cid, now_utc()))
        self.db.commit()

```

Idempotency store.

```python
    async def skip_duplicate(self, msg, cid):
        when = self.store.done_at(self.name, cid)
        if self.args.no_idempotency or not cid or not when:
            return False
        alog(
            self.role,
            "DUPLICATE_SKIPPED",
            cid,
            offset=msg.offset,
            first_processed_at=when,
            note="no LLM call, nothing produced",
        )
        await self.commit(msg, "duplicate skipped")
        return True

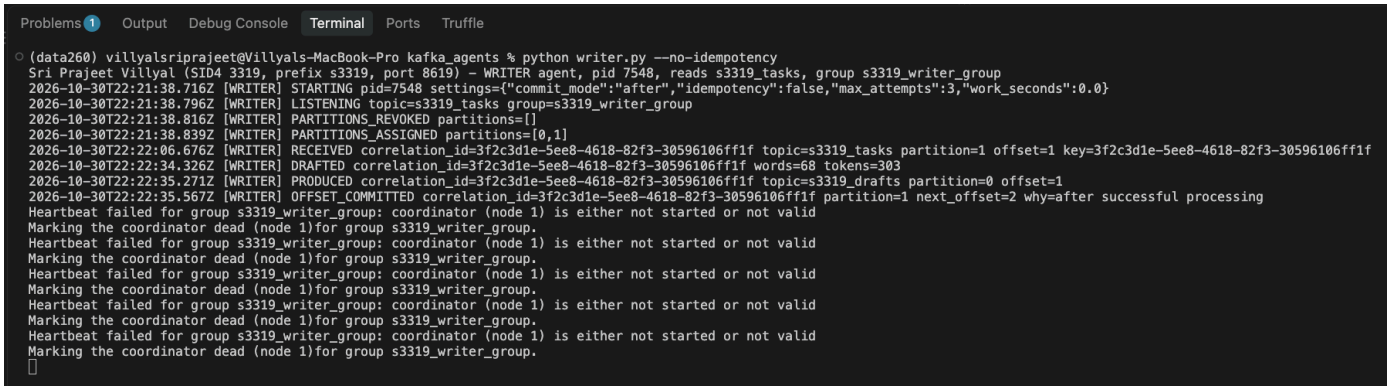
```

Skip a correlation_id that is already done.

Before (writer.py --no-idempotency):

```
(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents % cd ~/Desktop/data260-3319/code/kafka_agents
(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents % python replay_message.py --cid 3f2c3d1e-5ee8-4618-82f3-30596106ff1f
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) - Part 3 duplicate replay of 3f2c3d1e-5ee8-4618-82f3-30596106ff1f on s3319_tasks
2026-10-30T22:22:06.572Z [REPLAY] ORIGINAL_FOUND correlation_id=3f2c3d1e-5ee8-4618-82f3-30596106ff1f topic=s3319_tasks partition=1 offset=0
2026-10-30T22:22:06.667Z [REPLAY] DUPLICATE_SENT correlation_id=3f2c3d1e-5ee8-4618-82f3-30596106ff1f topic=s3319_tasks partition=1 offset=1 note=same key, same value, same correlation_id
(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents %
```

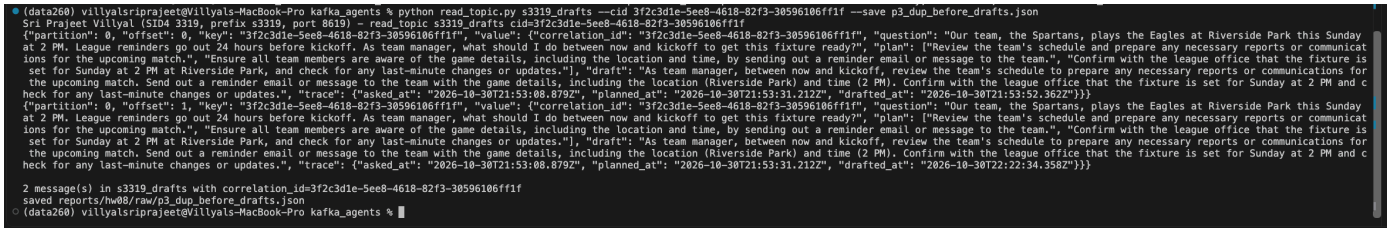
Replay sent.



Before: Writer processed the replay again.



Reviewer skipped the extra draft.

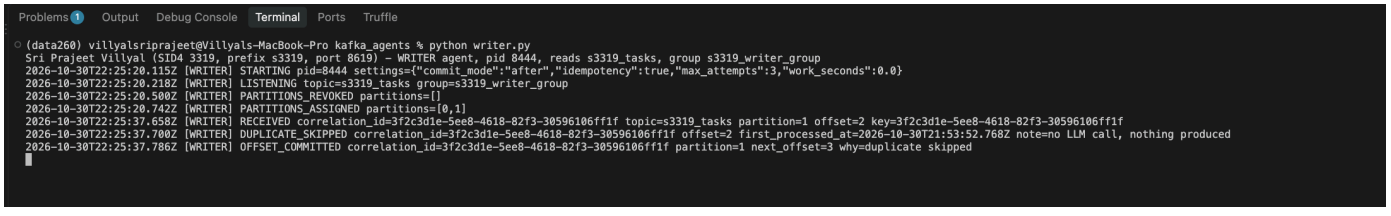


Before: 2 drafts for one question.

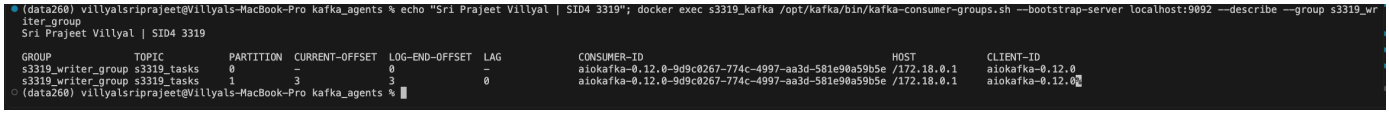
After (default writer.py):



Second replay sent.



After: DUPLICATE_SKIPPED, offset committed.



After: lag 0.



After: still 2 drafts and 1 final.



Duplicate test journey.

II. Consumer death

The --work-seconds 60 flag adds a slow step before the LLM call so I had time to kill -9 the Writer in the middle of its work.

```python
async def make_consumer(topic, group, role):
    consumer = AIOKafkaConsumer(
        bootstrap_servers=KAFKA_BOOTSTRAP,
        group_id=group,
        auto_offset_reset="earliest",
        enable_auto_commit=False,  # offsets are committed by hand
        session_timeout_ms=10000,
        heartbeat_interval_ms=3000,
    )
    await consumer.start()
    consumer.subscribe([topic], listener=LogAssignments(role))
    return consumer

```

Auto commit off.

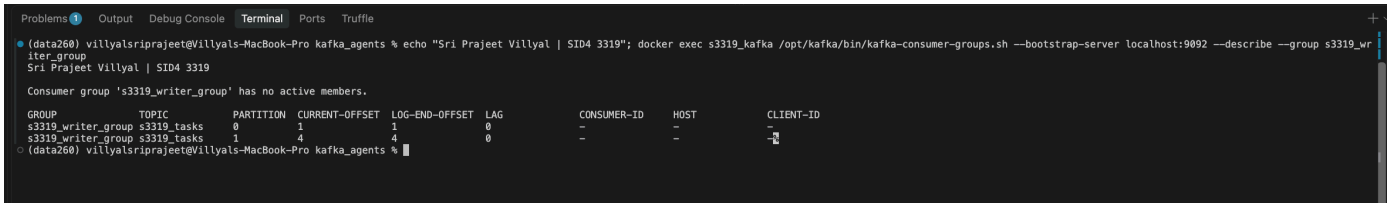
```python
    async def handle(self, msg):
        record = decode(msg.value)
        cid = record.get("correlation_id") if record else None
        alog(
            self.role,
            "RECEIVED",
            cid,
            topic=msg.topic,
            partition=msg.partition,
            offset=msg.offset,
            key=msg.key.decode("utf-8", errors="replace") if msg.key else None,
        )
        if self.args.commit_mode == "before":
            await self.commit(msg, "BEFORE processing (at-most-once)")
        if record and record.get("type") != "ordering_test" and await self.skip_duplicate(msg, cid):
            return
        if cid and (earlier := self.store.mark_started(self.name, cid)):
            alog(
                self.role,
                "REDELIVERED",
                cid,
                offset=msg.offset,
                first_started_at=earlier,
                note="earlier attempt never finished",
            )
        outputs = await self.run_with_retries(msg, record)
        if outputs is None:
            await self.commit(msg, "after dead letter")
            return
        for topic, key, value in outputs:
            await self.send(topic, key, value)
        if cid:
            self.store.mark_done(self.name, cid)
        if self.args.commit_mode == "after":
            await self.commit(msg, "after successful processing")
```

Commit only after the output is sent.

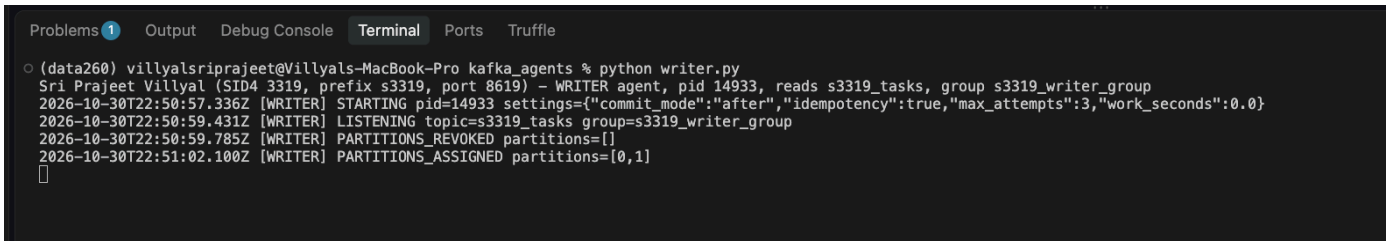
Before (writer.py --commit-mode before --work-seconds 60):



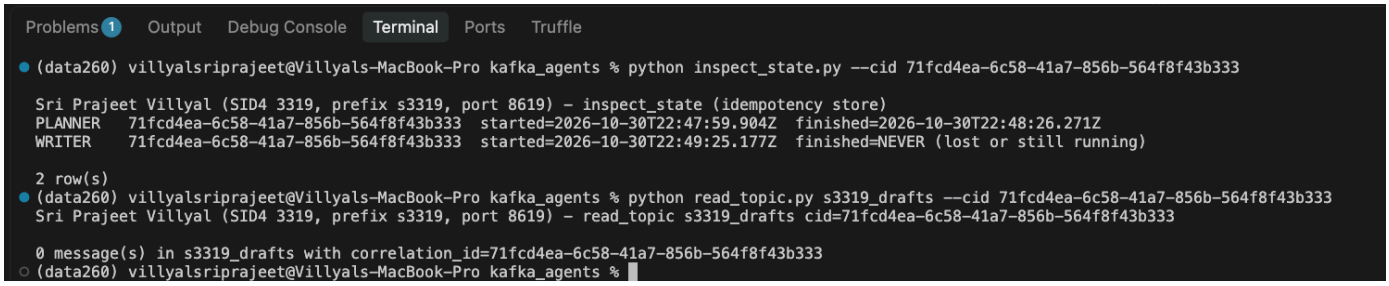
Before: offset committed on receipt, then killed.



Before: lag 0, Kafka thinks it is done.

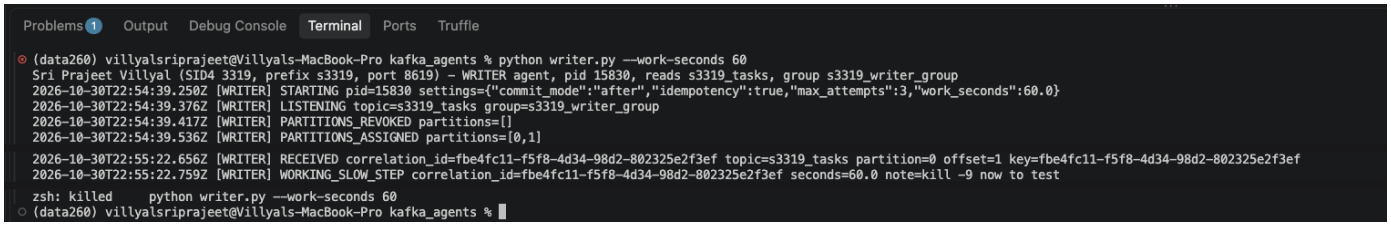


Before: restarted Writer gets nothing.



Before: never finished, 0 drafts. Lost.

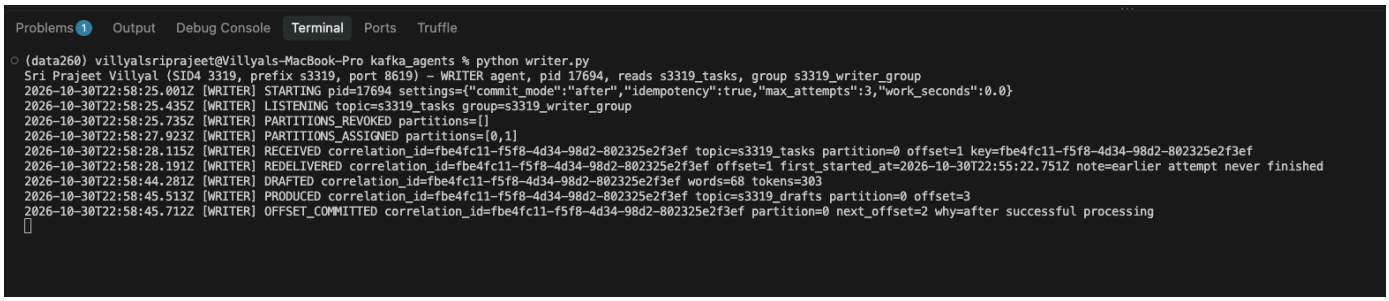
After (writer.py --work-seconds 60, commit after):



After: killed with no commit.



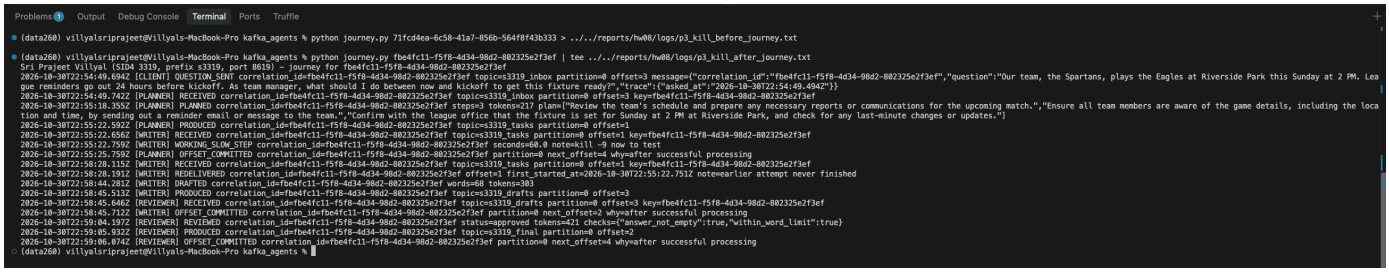
After: lag 1, task still waiting.



After: REDELIVERED and completed.



After: all agents finished, 1 draft.



Consumer death journey.

III. Poison message



Creating s3319_dead_letter.

```python
KEY = b"poison-demo"  # same key, same partition: the good task waits behind it
POISON = b'{"correlation_id": "poison-0001", "plan": [ <<< not valid JSON'
FOLLOW_UP_QUESTION = (
    "The Spartans' next fixture is Sunday at 2 PM at Riverside Park. What should the captain remind the team?"
)


async def main():
    show_banner("Part 3 send_poison.py")
    good_cid = str(uuid.uuid4())
    good = {
        "correlation_id": good_cid,
        "type": "answer_task",
        "question": FOLLOW_UP_QUESTION,
        "plan": ["State the fixture time and venue.", "List what players should bring.", "Add a reminder."],
        "trace": {"asked_at": now_utc(), "planned_at": now_utc()},
    }
    producer = await make_producer()
    bad = await producer.send_and_wait(TASKS, POISON, key=KEY)
    alog("POISON", "POISON_SENT", "poison-0001", topic=TASKS, partition=bad.partition, offset=bad.offset)
    ok = await producer.send_and_wait(TASKS, encode(good), key=KEY)
    alog("POISON", "GOOD_TASK_SENT_BEHIND_IT", good_cid, topic=TASKS, partition=ok.partition, offset=ok.offset)
    await producer.stop()
    info = {
        "poison": {"partition": bad.partition, "offset": bad.offset, "value": POISON.decode()},
        "follow_up": {"correlation_id": good_cid, "partition": ok.partition, "offset": ok.offset},
    }
    RAW_DIR.mkdir(parents=True, exist_ok=True)
    (RAW_DIR / "p3_poison_sent.json").write_text(json.dumps(info, indent=2) + "\n")
    print(json.dumps(info, indent=2))
```

send_poison.py: bad message, then a good one.

```python
    async def dead_letter(self, msg, record, attempts, error):
        cid = record.get("correlation_id") if record else None
        entry = {
            "correlation_id": cid,
            "failed_agent": self.role,
            "source_topic": msg.topic,
            "source_partition": msg.partition,
            "source_offset": msg.offset,
            "attempts": attempts,
            "error": error,
            "original_value": msg.value.decode("utf-8", errors="replace"),
            "failed_at": now_utc(),
        }
        key = msg.key.decode("utf-8", errors="replace") if msg.key else cid
        await self.send(DEAD_LETTER, key, entry)
        alog(self.role, "MOVED_TO_DEAD_LETTER", cid, topic=DEAD_LETTER, attempts=attempts, error=error)

    async def run_with_retries(self, msg, record):
        attempt = 0
        while True:
            attempt += 1
            try:
                if record is None:
                    raise ValueError("message is not a JSON object")
                return await self.handler(record, msg, self)
            except Exception as exc:  # any failure is retried, then dead-lettered
                error = f"{type(exc).__name__}: {exc}"
                limit = self.max_attempts or "no limit"
                alog(
                    self.role,
                    "PROCESSING_FAILED",
                    record.get("correlation_id") if record else None,
                    offset=msg.offset,
                    attempt=f"{attempt}/{limit}",
                    error=error,
                )
                if self.max_attempts and attempt >= self.max_attempts:
                    await self.dead_letter(msg, record, attempt, error)
                    return None
                await asyncio.sleep(min(2 ** (attempt - 1), 4))  # backoff 1s, 2s, 4s
```

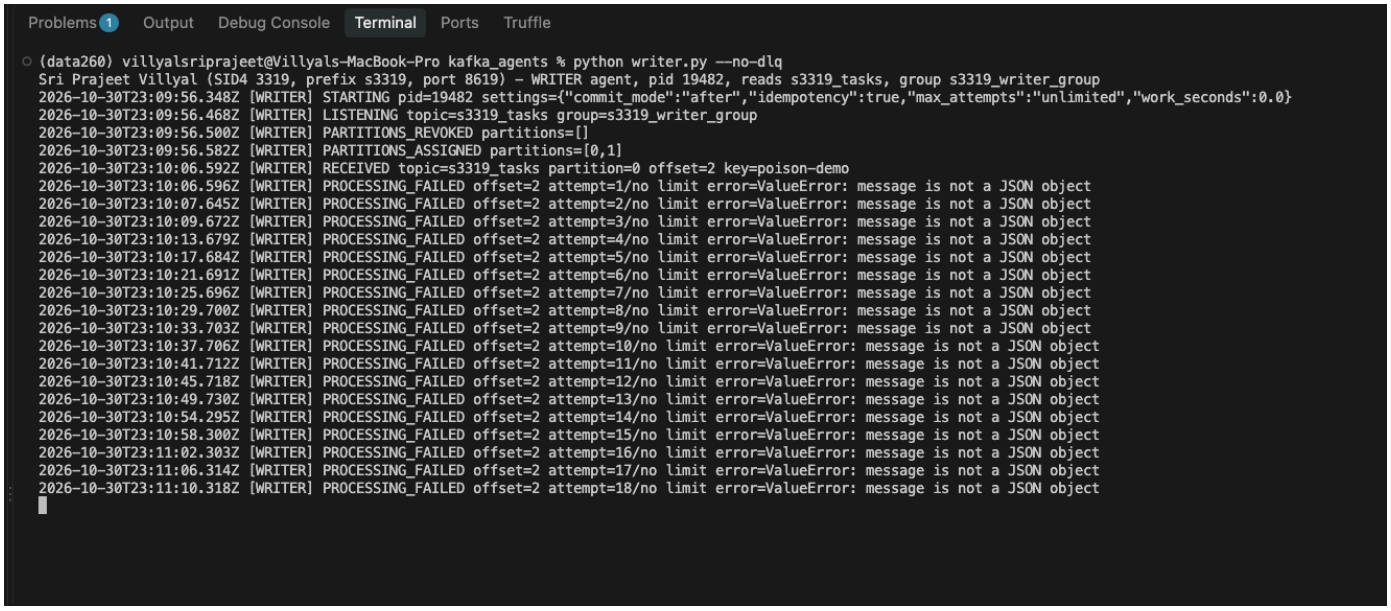
3 retries, then dead letter.

```
Problems 1  Output  Debug Console  Terminal  Ports  Truffle

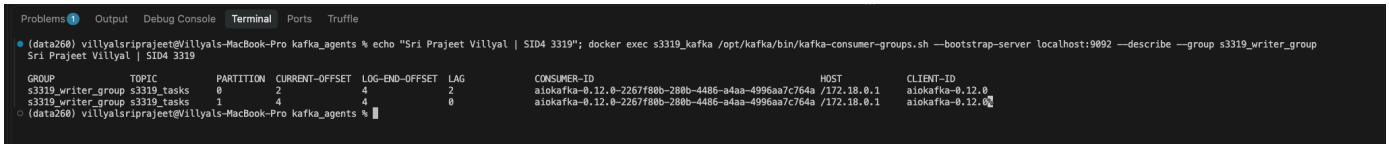
(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents % python send_poison.py
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) - Part 3 send_poison.py
2026-10-30T23:10:06.542Z [POISON] POISON_SENT correlation_id=poison-0001 topic=s3319_tasks partition=0 offset=2
2026-10-30T23:10:06.547Z [POISON] GOOD_TASK_SENT_BEHIND_IT correlation_id=f9b537df-be27-42a1-9a8d-4488a3daf7e2 topic=s3319_tasks partition=0 offset=3
{
  "poison": {
    "partition": 0,
    "offset": 2,
    "value": "{\"correlation_id\": \"poison-0001\", \"plan\": [ <<< not valid JSON"
  },
  "follow_up": {
    "correlation_id": "f9b537df-be27-42a1-9a8d-4488a3daf7e2",
    "partition": 0,
    "offset": 3
  }
}
(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents %
```

Poison at offset 2, good task at offset 3.

Before (writer.py --no-dlq):

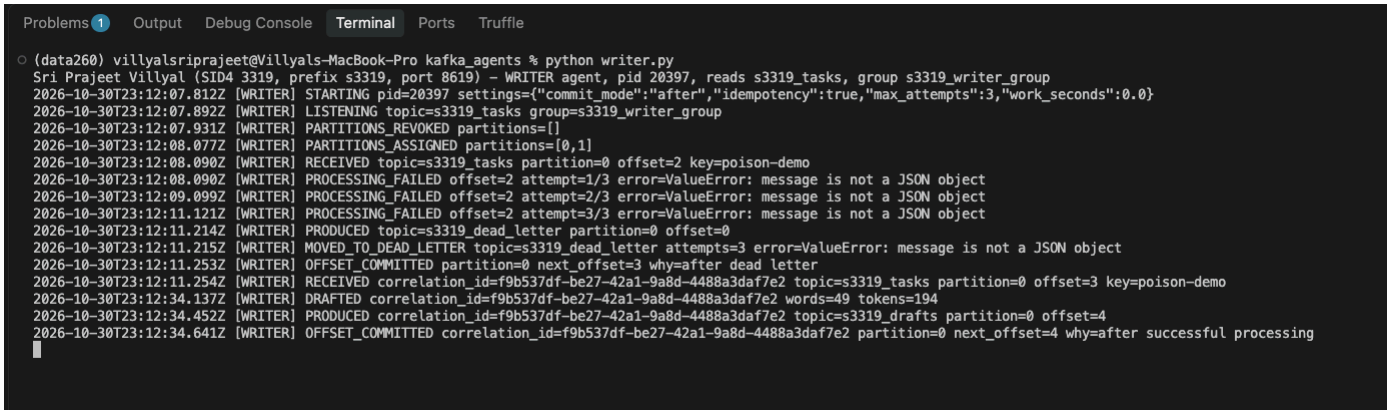


Before: endless retries.



Before: lag 2, partition blocked.

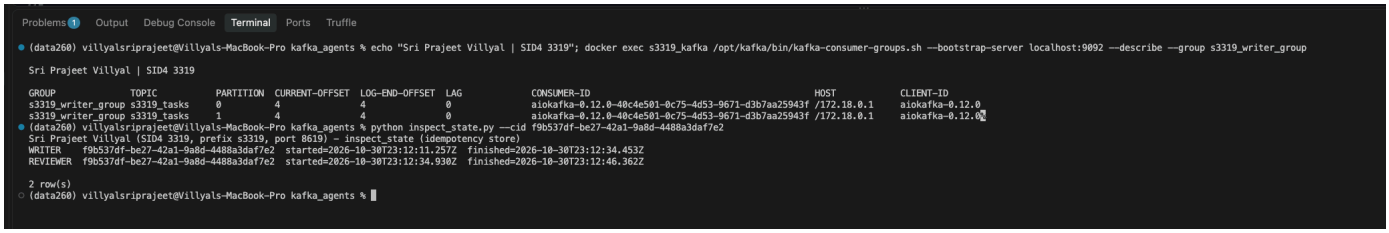
After (default writer.py):



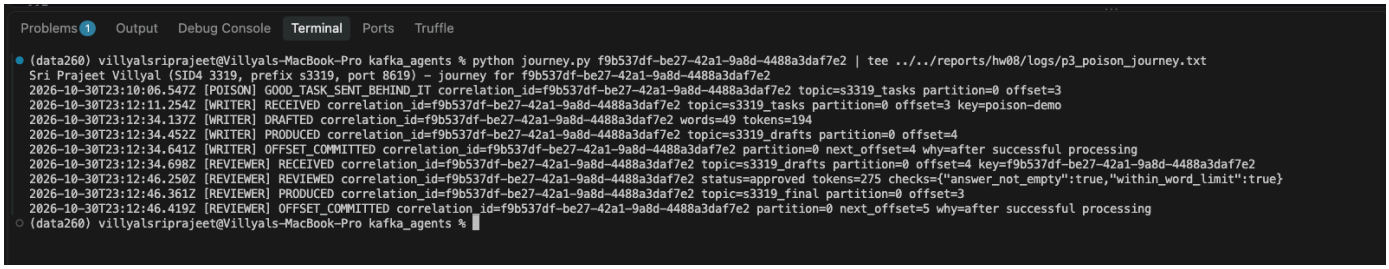
After: 3 attempts, dead letter, good task runs.



Dead letter entry.



After: lag 0, good task finished.



Good task journey.

IV. Ordering

ordering_test.py builds 20 related messages from VERIFY_SEED 263319: 4 fixtures with 5 updates each, in order FIXTURE_CREATED, VENUE_CONFIRMED, KICKOFF_SET, TICKETS_RELEASED, FIXTURE_CONFIRMED. They go to s3319_tasks and two Writers in the same group read them.

```
def build_messages(run):
    rng = random.Random(ORDER_SEED)
    codes = sorted({f"FX-{rng.randint(10000, 99999)}" for _ in range(20)})[:N_FIXTURES]
    queues = {code: list(range(1, len(EVENTS) + 1)) for code in codes}
    messages = []
    while any(queues.values()):
        code = rng.choice([c for c in codes if queues[c]])  # interleave, keep each fixture's own order
        seq = queues[code].pop(0)
        msg_no = len(messages) + 1
        cid = str(uuid.uuid5(uuid.NAMESPACE_URL, f"s3319-{run}-{ORDER_SEED}-{msg_no}"))
        messages.append(
            {
                "correlation_id": cid,
                "type": "ordering_test",
                "run": run,
                "msg_no": msg_no,
                "entity": code,
                "seq": seq,
                "event": EVENTS[seq - 1],
                "work_ms": rng.randint(50, 600),
            }
        )
    return messages

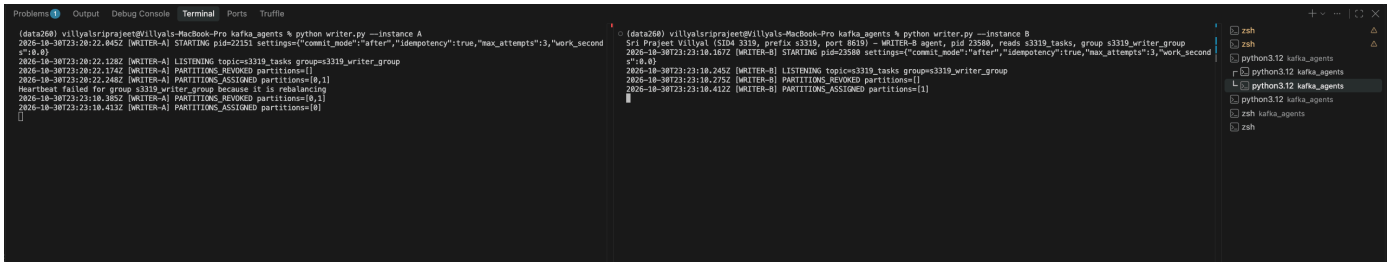
```

20 messages from VERIFY_SEED.

```python
async def send_all(run, keyed):
    random.seed(ORDER_SEED)  # the no-key partitioner picks randomly; seeding makes it repeatable
    producer = await make_producer()
    sent = []
    for msg in build_messages(run):
        key = msg["entity"].encode() if keyed else None  # keyed run: key = related fixture
        meta = await producer.send_and_wait(TASKS, encode(msg), key=key)
        sent.append(
            {
                **msg,
                "key": msg["entity"] if keyed else None,
                "partition": meta.partition,
                "offset": meta.offset,
                "sent_at": now_utc(),
            }
        )
        alog(
            "ORDER-TEST",
            "SENT",
            msg["correlation_id"],
            run=run,
            msg_no=msg["msg_no"],
            entity=msg["entity"],
            seq=msg["seq"],
            key=sent[-1]["key"],
            partition=meta.partition,
        )
    await producer.stop()
    return sent
```

Partition key .

Before (no key):



Two Writers, one partition each.

```
Problems 1  Output  Debug Console  Terminal  Ports  Truffle

(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents % python ordering_test.py nokey
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) - Part 3 ordering test, run=nokey
2026-10-30T23:23:52.578Z [ORDER-TEST] SENT correlation_id=db491947-7871-5dc4-8a1b-244df6771285 run=nokey msg_no=1 entity=FX-13344 seq=1 key=None partition=0
2026-10-30T23:23:52.581Z [ORDER-TEST] SENT correlation_id=1e22e026-5e30-5746-a888-096ecd33b0e4 run=nokey msg_no=2 entity=FX-13355 seq=1 key=None partition=0
2026-10-30T23:23:52.585Z [ORDER-TEST] SENT correlation_id=cda5a9ea-9686-5ad2-8e71-a273345606f6 run=nokey msg_no=3 entity=FX-30687 seq=1 key=None partition=1
2026-10-30T23:23:52.588Z [ORDER-TEST] SENT correlation_id=c098f2f2-e84c-5d16-b954-5f9f010c83c7 run=nokey msg_no=4 entity=FX-13344 seq=2 key=None partition=1
2026-10-30T23:23:52.598Z [ORDER-TEST] SENT correlation_id=0a3b104a-8c78-591b-a735-6077989d3d95 run=nokey msg_no=5 entity=FX-15596 seq=1 key=None partition=1
2026-10-30T23:23:52.602Z [ORDER-TEST] SENT correlation_id=71d2d916-39d8-5682-a16e-d4430e4caff3 run=nokey msg_no=6 entity=FX-15596 seq=2 key=None partition=0
2026-10-30T23:23:52.613Z [ORDER-TEST] SENT correlation_id=b66e482e-fec9-5e07-a360-9ac3c5bb610f run=nokey msg_no=7 entity=FX-13355 seq=2 key=None partition=1
2026-10-30T23:23:52.620Z [ORDER-TEST] SENT correlation_id=50b390cf-05f8-5f46-be14-c0515a4bf0f3 run=nokey msg_no=8 entity=FX-13355 seq=3 key=None partition=1
2026-10-30T23:23:52.627Z [ORDER-TEST] SENT correlation_id=57de1cb6-4022-5ffc-8b31-9547fb26d207 run=nokey msg_no=9 entity=FX-13344 seq=3 key=None partition=0
2026-10-30T23:23:52.634Z [ORDER-TEST] SENT correlation_id=0998b629-573c-5f46-a68a-c4261f67ffb6 run=nokey msg_no=10 entity=FX-30687 seq=2 key=None partition=0
2026-10-30T23:23:52.638Z [ORDER-TEST] SENT correlation_id=8039202f-ae6d-538b-afad-f105236711a6 run=nokey msg_no=11 entity=FX-13355 seq=4 key=None partition=0
2026-10-30T23:23:52.643Z [ORDER-TEST] SENT correlation_id=a77f9122-2c88-5ab4-8cab-0665feaf54bc run=nokey msg_no=12 entity=FX-13344 seq=4 key=None partition=1
2026-10-30T23:23:52.647Z [ORDER-TEST] SENT correlation_id=6caf9e51-f393-5084-9264-ad3b7fab9648 run=nokey msg_no=13 entity=FX-15596 seq=3 key=None partition=0
2026-10-30T23:23:52.652Z [ORDER-TEST] SENT correlation_id=92e15823-46ae-5ef7-ae33-d60aefc310a3 run=nokey msg_no=14 entity=FX-15596 seq=4 key=None partition=0
2026-10-30T23:23:52.655Z [ORDER-TEST] SENT correlation_id=40a2ed7a-6f09-561b-838e-48a34331a876 run=nokey msg_no=15 entity=FX-15596 seq=5 key=None partition=0
2026-10-30T23:23:52.657Z [ORDER-TEST] SENT correlation_id=d6853ce0-2f37-5bb5-91e0-634253e7da5c run=nokey msg_no=16 entity=FX-30687 seq=3 key=None partition=0
2026-10-30T23:23:52.659Z [ORDER-TEST] SENT correlation_id=b5c1d328-3256-55d0-b4dc-e5bc257bcbc1 run=nokey msg_no=17 entity=FX-30687 seq=4 key=None partition=1
2026-10-30T23:23:52.660Z [ORDER-TEST] SENT correlation_id=bd25f5a0-6cba-54c8-80cd-900147faaca0 run=nokey msg_no=18 entity=FX-13344 seq=5 key=None partition=1
2026-10-30T23:23:52.661Z [ORDER-TEST] SENT correlation_id=cea2402d-2665-55f4-87b8-534b6a6b389a run=nokey msg_no=19 entity=FX-30687 seq=5 key=None partition=0
2026-10-30T23:23:52.663Z [ORDER-TEST] SENT correlation_id=3a41fec0-1381-5e14-812b-c192e2640d0f run=nokey msg_no=20 entity=FX-13355 seq=5 key=None partition=0

processing order (both Writers, by time):
  2026-10-30T23:23:52.794Z  WRITER-A  p0 off=4    # 1 FX-13344 seq 1
  2026-10-30T23:23:53.039Z  WRITER-B  p1 off=4    # 3 FX-30687 seq 1
  2026-10-30T23:23:53.330Z  WRITER-B  p1 off=5    # 4 FX-13344 seq 2
  2026-10-30T23:23:53.554Z  WRITER-A  p0 off=5    # 2 FX-13355 seq 1
  2026-10-30T23:23:53.721Z  WRITER-B  p1 off=6    # 5 FX-15596 seq 1
  2026-10-30T23:23:54.131Z  WRITER-A  p0 off=6    # 6 FX-15596 seq 2
  2026-10-30T23:23:54.232Z  WRITER-A  p0 off=7    # 9 FX-13344 seq 3
  2026-10-30T23:23:54.311Z  WRITER-B  p1 off=7    # 7 FX-13355 seq 2
  2026-10-30T23:23:54.443Z  WRITER-A  p0 off=8    #10 FX-30687 seq 2
  2026-10-30T23:23:54.775Z  WRITER-A  p0 off=9    #11 FX-13355 seq 4
  2026-10-30T23:23:54.906Z  WRITER-B  p1 off=8    # 8 FX-13355 seq 3
  2026-10-30T23:23:55.021Z  WRITER-A  p0 off=10   #13 FX-15596 seq 3
  2026-10-30T23:23:55.334Z  WRITER-B  p1 off=9    #12 FX-13344 seq 4
  2026-10-30T23:23:55.498Z  WRITER-A  p0 off=11   #14 FX-15596 seq 4
  2026-10-30T23:23:55.629Z  WRITER-A  p0 off=12   #15 FX-15596 seq 5
  2026-10-30T23:23:55.763Z  WRITER-B  p1 off=10   #17 FX-30687 seq 4
  2026-10-30T23:23:56.022Z  WRITER-B  p1 off=11   #18 FX-13344 seq 5
  2026-10-30T23:23:56.248Z  WRITER-A  p0 off=13   #16 FX-30687 seq 3
  2026-10-30T23:23:56.364Z  WRITER-A  p0 off=14   #19 FX-30687 seq 5
  2026-10-30T23:23:56.738Z  WRITER-A  p0 off=15   #20 FX-13355 seq 5

Ordering run 'nokey' (VERIFY_SEED 263319)
fixture    partitions  writers             processed seq order  in order?
FX-13344   [0, 1]      WRITER-A, WRITER-B   [1, 2, 3, 4, 5]      YES
FX-13355   [0, 1]      WRITER-A, WRITER-B   [1, 2, 4, 3, 5]      NO
FX-15596   [0, 1]      WRITER-A, WRITER-B   [1, 2, 3, 4, 5]      YES
FX-30687   [0, 1]      WRITER-A, WRITER-B   [1, 2, 4, 3, 5]      NO

messages processed: 20/20
fixtures kept in one partition: 0/4
fixtures processed in order: 2/4
(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents %
```

No key: 0/4 in one partition, 2/4 in order.

```
Problems 1  Output  Debug Console  Terminal  Ports  Truffle

(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents % echo "Sri Prajeet Villyal | SID4 3319"; grep -E "T23:23:5[2-9].*ORDERING_PROCESSED" ../../reports/hw08/logs/agents.log
Sri Prajeet Villyal | SID4 3319
2026-10-30T23:23:52.800Z [WRITER-A] ORDERING_PROCESSED correlation_id=db491947-7871-5dc4-8a1b-244df6771285 entity=FX-13344 seq=1 partition=0 offset=4
2026-10-30T23:23:53.043Z [WRITER-B] ORDERING_PROCESSED correlation_id=cda5a9ea-9686-5ad2-8e71-a273345606f6 entity=FX-30687 seq=1 partition=1 offset=4
2026-10-30T23:23:53.331Z [WRITER-B] ORDERING_PROCESSED correlation_id=c098f2f2-e84c-5d16-b954-5f9f010c83c7 entity=FX-13344 seq=2 partition=1 offset=5
2026-10-30T23:23:53.558Z [WRITER-A] ORDERING_PROCESSED correlation_id=1e22e026-5e30-5746-a888-096ecd33b0e4 entity=FX-13355 seq=1 partition=0 offset=5
2026-10-30T23:23:53.724Z [WRITER-B] ORDERING_PROCESSED correlation_id=0a3b104a-8c78-591b-a735-6077989d3d95 entity=FX-15596 seq=1 partition=1 offset=6
2026-10-30T23:23:54.134Z [WRITER-A] ORDERING_PROCESSED correlation_id=71d2d916-39d8-5682-a16e-d4430e4caff3 entity=FX-15596 seq=2 partition=0 offset=6
2026-10-30T23:23:54.233Z [WRITER-A] ORDERING_PROCESSED correlation_id=57de1cb6-4022-5ffc-8b31-9547fb26d207 entity=FX-13344 seq=3 partition=0 offset=7
2026-10-30T23:23:54.312Z [WRITER-B] ORDERING_PROCESSED correlation_id=b66e482e-fec9-5e07-a360-9ac3c5bb610f entity=FX-13355 seq=2 partition=1 offset=7
2026-10-30T23:23:54.444Z [WRITER-A] ORDERING_PROCESSED correlation_id=0998b629-573c-5f46-a68a-c4261f67ffb6 entity=FX-30687 seq=2 partition=0 offset=8
2026-10-30T23:23:54.777Z [WRITER-A] ORDERING_PROCESSED correlation_id=8039202f-ae6d-538b-afad-f105236711a6 entity=FX-13355 seq=4 partition=0 offset=9
2026-10-30T23:23:54.907Z [WRITER-B] ORDERING_PROCESSED correlation_id=50b390cf-05f8-5f46-be14-c0515a4bf0f3 entity=FX-13355 seq=3 partition=1 offset=8
2026-10-30T23:23:55.022Z [WRITER-A] ORDERING_PROCESSED correlation_id=6caf9e51-f393-5084-9264-ad3b7fab9648 entity=FX-15596 seq=3 partition=0 offset=10
2026-10-30T23:23:55.336Z [WRITER-B] ORDERING_PROCESSED correlation_id=a77f9122-2c88-5ab4-8cab-0665feaf54bc entity=FX-13344 seq=4 partition=1 offset=9
2026-10-30T23:23:55.499Z [WRITER-A] ORDERING_PROCESSED correlation_id=92e15823-46ae-5ef7-ae33-d60aefc310a3 entity=FX-15596 seq=4 partition=0 offset=11
2026-10-30T23:23:55.631Z [WRITER-A] ORDERING_PROCESSED correlation_id=40a2ed7a-6f09-561b-838e-48a34331a876 entity=FX-15596 seq=5 partition=0 offset=12
2026-10-30T23:23:55.764Z [WRITER-B] ORDERING_PROCESSED correlation_id=b5c1d328-3256-55d0-b4dc-e5bc257bcbc1 entity=FX-30687 seq=4 partition=1 offset=10
2026-10-30T23:23:56.023Z [WRITER-B] ORDERING_PROCESSED correlation_id=bd25f5a0-6cba-54c8-80cd-900147faaca0 entity=FX-13344 seq=5 partition=1 offset=11
2026-10-30T23:23:56.260Z [WRITER-A] ORDERING_PROCESSED correlation_id=d6853ce0-2f37-5bb5-91e0-634253e7da5c entity=FX-30687 seq=3 partition=0 offset=13
2026-10-30T23:23:56.365Z [WRITER-A] ORDERING_PROCESSED correlation_id=cea2402d-2665-55f4-87b8-534b6a6b389a entity=FX-30687 seq=5 partition=0 offset=14
2026-10-30T23:23:56.752Z [WRITER-A] ORDERING_PROCESSED correlation_id=3a41fec0-1381-5e14-812b-c192e2640d0f entity=FX-13355 seq=5 partition=0 offset=15
(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents %
```

No key: fixtures split across both Writers.

After (key = fixture code):

```
(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents % python writer.py --instance B
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) — WRITER-B agent, pid 25316, reads s3319_tasks, group s3319_writer_group
2026-10-30T23:32:38.465Z [WRITER-B] STARTING pid=25316 settings={"commit_mode":"after","idempotency":true,"max_attempts":3,"work_second
s":0.0}
2026-10-30T23:32:38.709Z [WRITER-B] LISTENING topic=s3319_tasks group=s3319_writer_group
2026-10-30T23:32:39.522Z [WRITER-B] PARTITIONS_REVOKED partitions=[]
2026-10-30T23:32:41.101Z [WRITER-B] PARTITIONS_ASSIGNED partitions=[1]
```

Writer B on partition 1.

```
Problems 1  Output  Debug Console  Terminal  Ports  Truffle

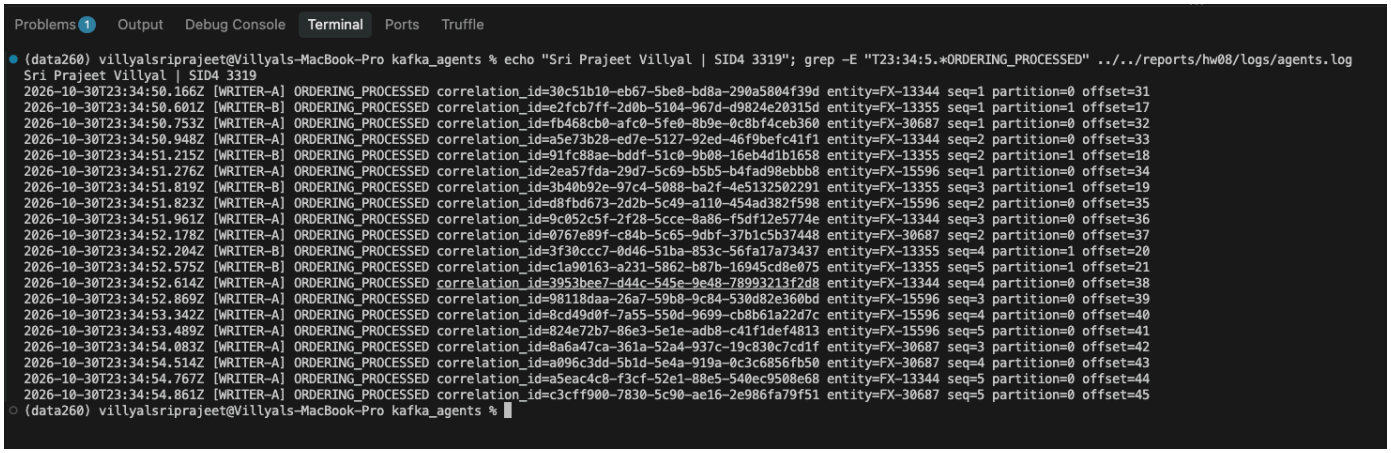
(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents % python ordering_test.py keyed
Sri Prajeet Villyal (SID4 3319, prefix s3319, port 8619) — Part 3 ordering test, run=keyed
2026-10-30T23:34:49.927Z [ORDER-TEST] SENT correlation_id=30c51b10-eb67-5be8-bd8a-290a5804f39d run=keyed msg_no=1 entity=FX-13344 seq=1 key=FX-13344 partition=0
2026-10-30T23:34:49.937Z [ORDER-TEST] SENT correlation_id=e2fcb7ff-2d0b-5104-967d-d9824e20315d run=keyed msg_no=2 entity=FX-13355 seq=1 key=FX-13355 partition=1
2026-10-30T23:34:49.941Z [ORDER-TEST] SENT correlation_id=fb468cb0-afc0-5fe0-8b9e-0c8bf4ceb360 run=keyed msg_no=3 entity=FX-30687 seq=1 key=FX-30687 partition=0
2026-10-30T23:34:49.948Z [ORDER-TEST] SENT correlation_id=a5e73b28-ed7e-5127-92ed-46f9befc41f1 run=keyed msg_no=4 entity=FX-13344 seq=2 key=FX-13344 partition=0
2026-10-30T23:34:49.953Z [ORDER-TEST] SENT correlation_id=2ea57fda-29d7-5c69-b5b5-b4fad98ebbb8 run=keyed msg_no=5 entity=FX-15596 seq=1 key=FX-15596 partition=0
2026-10-30T23:34:49.961Z [ORDER-TEST] SENT correlation_id=d8fbd673-2d2b-5c49-a110-454ad382f598 run=keyed msg_no=6 entity=FX-15596 seq=2 key=FX-15596 partition=0
2026-10-30T23:34:49.967Z [ORDER-TEST] SENT correlation_id=91fc88ae-bddf-51c0-9b08-16eb4d1b1658 run=keyed msg_no=7 entity=FX-13355 seq=2 key=FX-13355 partition=1
2026-10-30T23:34:49.969Z [ORDER-TEST] SENT correlation_id=3b40b92e-97c4-5088-ba2f-4e5132502291 run=keyed msg_no=8 entity=FX-13355 seq=3 key=FX-13355 partition=1
2026-10-30T23:34:49.974Z [ORDER-TEST] SENT correlation_id=9c052c5f-2f28-5cce-8a86-f5df12e5774e run=keyed msg_no=9 entity=FX-13344 seq=3 key=FX-13344 partition=0
2026-10-30T23:34:49.977Z [ORDER-TEST] SENT correlation_id=0767e89f-c84b-5c65-9dbf-37b1c5b37448 run=keyed msg_no=10 entity=FX-30687 seq=2 key=FX-30687 partition=0
2026-10-30T23:34:49.984Z [ORDER-TEST] SENT correlation_id=3f30ccc7-0d46-51ba-853c-56fa17a73437 run=keyed msg_no=11 entity=FX-13355 seq=4 key=FX-13355 partition=1
2026-10-30T23:34:49.988Z [ORDER-TEST] SENT correlation_id=3953bee7-d44c-545e-9e48-78993213f2d8 run=keyed msg_no=12 entity=FX-13344 seq=4 key=FX-13344 partition=0
2026-10-30T23:34:49.992Z [ORDER-TEST] SENT correlation_id=98118daa-26a7-59b8-9c84-530d82e360bd run=keyed msg_no=13 entity=FX-15596 seq=3 key=FX-15596 partition=0
2026-10-30T23:34:49.997Z [ORDER-TEST] SENT correlation_id=8cd49d0f-7a55-550d-9699-cb8b61a22d7c run=keyed msg_no=14 entity=FX-15596 seq=4 key=FX-15596 partition=0
2026-10-30T23:34:50.000Z [ORDER-TEST] SENT correlation_id=824e72b7-86e3-5e1e-adb8-c41f1def4813 run=keyed msg_no=15 entity=FX-15596 seq=5 key=FX-15596 partition=0
2026-10-30T23:34:50.003Z [ORDER-TEST] SENT correlation_id=8a6a47ca-361a-52a4-937c-19c830c7cd1f run=keyed msg_no=16 entity=FX-30687 seq=3 key=FX-30687 partition=0
2026-10-30T23:34:50.011Z [ORDER-TEST] SENT correlation_id=a096c3dd-5b1d-5e4a-919a-0c3c6856fb50 run=keyed msg_no=17 entity=FX-30687 seq=4 key=FX-30687 partition=0
2026-10-30T23:34:50.017Z [ORDER-TEST] SENT correlation_id=a5eac4c8-f3cf-52e1-88e5-540ec9508e68 run=keyed msg_no=18 entity=FX-13344 seq=5 key=FX-13344 partition=0
2026-10-30T23:34:50.032Z [ORDER-TEST] SENT correlation_id=c3cff900-7830-5c90-ae16-2e986fa79f51 run=keyed msg_no=19 entity=FX-30687 seq=5 key=FX-30687 partition=0
2026-10-30T23:34:50.057Z [ORDER-TEST] SENT correlation_id=c1a90163-a231-5862-b87b-16945cd8e075 run=keyed msg_no=20 entity=FX-13355 seq=5 key=FX-13355 partition=1

processing order (both Writers, by time):
  2026-10-30T23:34:50.162Z  WRITER-A  p0 off=31   # 1 FX-13344 seq 1
  2026-10-30T23:34:50.596Z  WRITER-B  p1 off=17   # 2 FX-13355 seq 1
  2026-10-30T23:34:50.749Z  WRITER-A  p0 off=32   # 3 FX-30687 seq 1
  2026-10-30T23:34:50.948Z  WRITER-A  p0 off=33   # 4 FX-13344 seq 2
  2026-10-30T23:34:51.214Z  WRITER-B  p1 off=18   # 7 FX-13355 seq 2
  2026-10-30T23:34:51.276Z  WRITER-A  p0 off=34   # 5 FX-15596 seq 1
  2026-10-30T23:34:51.814Z  WRITER-B  p1 off=19   # 8 FX-13355 seq 3
  2026-10-30T23:34:51.822Z  WRITER-A  p0 off=35   # 6 FX-15596 seq 2
  2026-10-30T23:34:51.960Z  WRITER-A  p0 off=36   # 9 FX-13344 seq 3
  2026-10-30T23:34:52.177Z  WRITER-A  p0 off=37   #10 FX-30687 seq 2
  2026-10-30T23:34:52.202Z  WRITER-B  p1 off=20   #11 FX-13355 seq 4
  2026-10-30T23:34:52.575Z  WRITER-B  p1 off=21   #20 FX-13355 seq 5
  2026-10-30T23:34:52.614Z  WRITER-A  p0 off=38   #12 FX-13344 seq 4
  2026-10-30T23:34:52.867Z  WRITER-A  p0 off=39   #13 FX-15596 seq 3
  2026-10-30T23:34:53.341Z  WRITER-A  p0 off=40   #14 FX-15596 seq 4
  2026-10-30T23:34:53.488Z  WRITER-A  p0 off=41   #15 FX-15596 seq 5
  2026-10-30T23:34:54.082Z  WRITER-A  p0 off=42   #16 FX-30687 seq 3
  2026-10-30T23:34:54.514Z  WRITER-A  p0 off=43   #17 FX-30687 seq 4
  2026-10-30T23:34:54.765Z  WRITER-A  p0 off=44   #18 FX-13344 seq 5
  2026-10-30T23:34:54.859Z  WRITER-A  p0 off=45   #19 FX-30687 seq 5

Ordering run 'keyed' (VERIFY_SEED 263319)
fixture    partitions  writers             processed seq order  in order?
FX-13344   [0]         WRITER-A            [1, 2, 3, 4, 5]      YES
FX-13355   [1]         WRITER-B            [1, 2, 3, 4, 5]      YES
FX-15596   [0]         WRITER-A            [1, 2, 3, 4, 5]      YES
FX-30687   [0]         WRITER-A            [1, 2, 3, 4, 5]      YES

messages processed: 20/20
fixtures kept in one partition: 4/4
fixtures processed in order: 4/4
(data260) villyalsriprajeet@Villyals-MacBook-Pro kafka_agents %
```

Keyed: 4/4 in one partition, 4/4 in order.



Keyed: each fixture stays with one Writer.

Failure table

Failure scenario	Before fix	After fix
Duplicate delivery	Replay at tasks p1 offset 1. Writer called the LLM again (303 tokens), drafts went from 1 to 2.	Replay at offset 2. Writer logged DUPLICATE_SKIPPED in 42 ms, no LLM call, drafts stayed at 2, finals at 1, lag 0.
Consumer killed mid-work	Offset committed on receipt. After kill -9 the lag was 0, the restart got nothing, the Writer never finished, 0 drafts (1 of 1 lost).	No commit before the work. kill -9 left lag 1, the restarted Writer got the same offset, logged REDELIVERED and finished in 17.6 s (0 lost).
Poison message	18 attempts in 64 s with no end. Partition 0 lag 2, the good task behind it never ran.	3 attempts in 3.0 s (backoff 1 s, 2 s), moved to s3319_dead_letter, good task picked up right after.
Two partitions / two consumers	No key: 0/4 fixtures in one partition, 2/4 in order (FX-13355 and FX-30687 ran as 1, 2, 4, 3, 5).	Key = fixture code: 4/4 in one partition, 4/4 in order. Writer A took p0 (15 messages) and Writer B took p1 (5 messages) in parallel.

Log excerpts for every row are the screenshots above, and the same lines are in reports/hw08/RUN_LOG.txt. The raw files (ordering CSVs and summaries, p3_dup_*_drafts.json, p3_dead_letter.json) are in reports/hw08/raw/.

V. Comparison with HW2

In HW2 the Planner and Reviewer were nodes in one LangGraph StateGraph. A supervisor routed between them, the state was a dict in memory, and each step waited on the one before it.

Benefits I gained with Kafka

1. Work survives a crash. Each handoff is a stored message with an offset, so when the Writer was killed the task was still in s3319_tasks and the next Writer finished it. In HW2 a crash lost the whole run because the state only lived in memory.

2. Agents scale and fail on their own. With two partitions I ran two Writers in one group and they shared the work in parallel. Each agent is its own process and can be restarted without stopping the others.

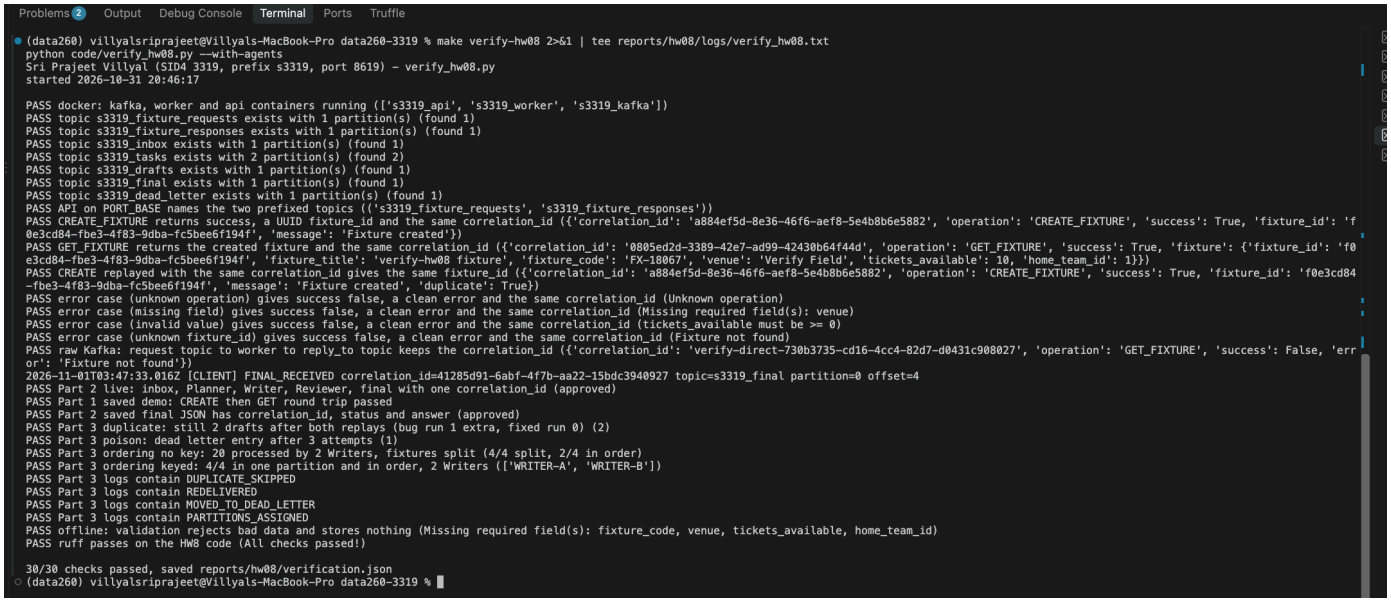
New responsibilities

1. Delivery rules are now my job. Kafka gives at least once delivery, so I had to add the idempotency store and commit only after processing. In HW2 a node ran once per call and none of this was needed.

2. Bad messages and ordering have to be handled on purpose. A poison message blocks its partition unless retries are bounded and there is a dead letter topic, and order is only kept inside one partition, so related messages need the same key. I also have to run the broker, create topics with the right partitions and watch consumer lag.

Verification

make verify-hw08 runs code/verify_hw08.py and writes reports/hw08/verification.json. It checks the containers and all 7 topics, the CREATE then GET round trip and error cases through FastAPI, the raw Kafka contract, one live question through the three agents, the saved Part 3 evidence and ruff. It ran on commit a364807 with **30/30 checks passed**. The hw8 tag is on the next commit, 456c240, which only adds verification.json and RUN_LOG.txt.



make verify-hw08: 30/30 passed.

AI Use

1. What I used AI for and what I did myself

I used an AI assistant as a helper for this homework. I wrote first drafts of the Kafka worker, the FastAPI wrapper, the three agents, the Part 3 test scripts and the verify script, and explained each step. I did the setup myself: Docker and the Kafka container, the conda environment, Ollama and the topics. I ran every command on my own laptop, started and killed the agents for the failure tests, took every screenshot, made all the commits and checked the numbers in METRICS.md against the logs and the raw files.

2. One AI output that was wrong

For the consumer death test the AI told me to start the buggy Writer with a 25 second slow step and kill it during that step. The window was too short to do by hand. The Planner took about 25 seconds first, and by the time I ran kill -9 the slow step was already over and the Writer had called the LLM and finished. So that before run did not show a lost message at all. I also checked one result myself: my first keyed ordering run showed 4/4 in order, but the writers column said only WRITER-A. Writer B had stopped, so the run did not test two consumers.

3. How I found it

After the kill I ran inspect_state.py and read_topic.py for that correlation_id. The Writer row had a finished time and s3319_drafts had 1 draft, so the task had completed and nothing was lost. For the ordering run, I read the summary table instead of only the 4/4 lines, and the Writer A terminal showed PARTITIONS_ASSIGNED [0,1].

4. What I changed and why it works now

I reran the test with --work-seconds 60 and killed the Writer as soon as WORKING_SLOW_STEP appeared, using pgrep so I did not have to type the pid. This time the offset was already committed, the lag was 0, the restarted Writer got nothing and there were 0 drafts, so the loss is real. The after run used the same 60 second step and the task was redelivered and completed. For ordering, I started Writer B again, waited until each Writer had one partition and reran the keyed test: 4/4 in one partition and 4/4 in order, with FX-13355 on Writer B and the other three on Writer A.